

DEEP LEARNING

Course Code: 57714

A Practical Journal Submitted in Fulfillment of the Degree of MASTER

In COMPUTER SCIENCE

Year 2025-2026

By

Ms. Jadhav Mansi Sanjay

(Application Id-

MU0027920240004612)

Seat No – 8174081

Semester-IV Under the

Guidance of



Centre for Distance and Online Education Vidya Nagari,
Kalina, Santacruz East – 400098. University of Mumbai

PCP Center

[Satish Pradhan Dyanasadhana College, Thane]



Institute of Distance and Open Learning
Vidya Nagari, Kalina, Santacruz East –400098.

CERTIFICATE

This is to certify that **Ms. Jadhav Mansi Sanjay**, appearing for the Master of Science (M.Sc.) in Computer Science (Semester IV), Application ID: MU0027920240004612, has satisfactorily completed the prescribed practical for **Deep Learning** (Course Code: 57714), as laid down by the University of Mumbai for the academic year 2025–26.

Teacher In Charge

External Examiner

Coordinator

Date :

Place :

Index

Sr.No	Practical Name	Page No	Signature
1	Implement Feed-forward Neural Network and train the network with different optimizers and compare the results		
2	Write a Program to implement regularization to prevent the model from overfitting		
3	Implement deep learning for recognizing classes for datasets like CIFAR-10 images for previously unseen images and assign them to one of the 10 classes		
4	Implement deep learning for the Prediction of the autoencoder from the test data (e.g. MNIST data set)		
5	Implement Convolutional Neural Network for Digit Recognition on the MNIST Dataset		
6	Write a program to implement Transfer Learning on the suitable dataset		
7	Write a program for the Implementation of a Generative Adversarial Network for generating synthetic shapes		
8	Write a program to implement a simple form of a recurrent neural network		
9	Implement LSTM for Sentiment Analysis on datasets like IMDB Movie Reviews Dataset or similar		
10	Write a program for object detection from the image/ video		

Practical 1: Implement Feed-forward Neural Network and train the network with different optimizers and compare the results

Aim

To implement a Feed Forward Neural Network using TensorFlow and train the model using different optimizers (SGD, RMSprop, and Adam) and compare their performance.

Solution:

Step 1: Import Required Libraries

The following libraries are required for building and training the neural network.

- TensorFlow – Deep Learning framework
- NumPy – Numerical computations
- Matplotlib – Data visualization

Step 2: Load the MNIST Dataset

The MNIST dataset contains handwritten digit images (0–9). It is commonly used for image classification tasks.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.21.0
NumPy Version: 2.4.6

```
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Display dataset information
print("Training data shape:", x_train.shape)
print("Training labels shape:", y_train.shape)
print("Testing data shape:", x_test.shape)
print("Testing labels shape:", y_test.shape)
```

Training data shape: (60000, 28, 28) Training labels
shape: (60000,) Testing data shape: (10000, 28, 28)
Testing labels shape: (10000,)

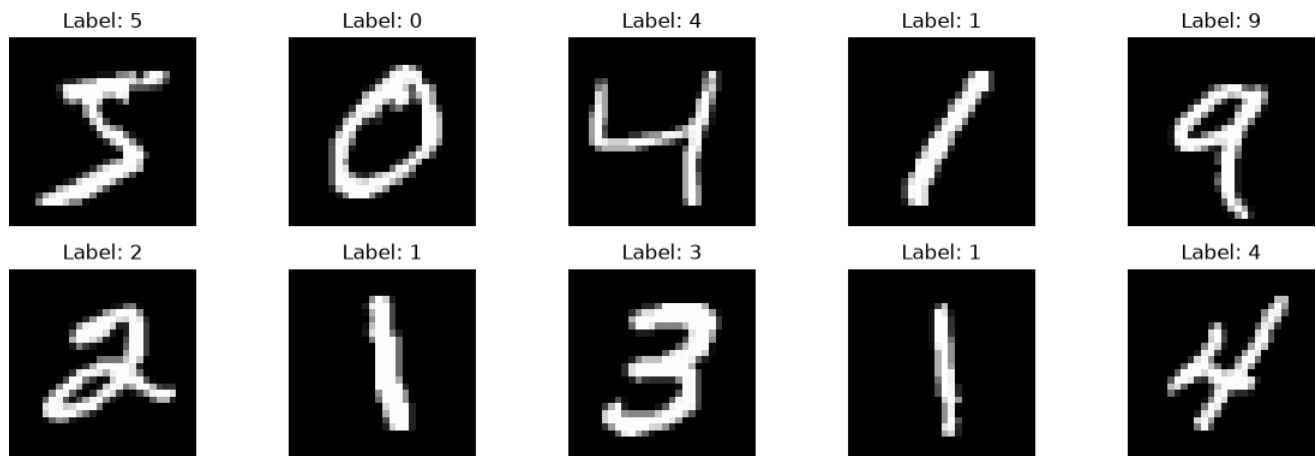
Step 3: Visualize Sample Images

Before training the model, it is useful to inspect a few images from the dataset to understand the input data.

```
# Display the first 10 images from the training dataset
plt.figure(figsize=(12, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 4: Data Preprocessing

Before training the neural network, the image data is preprocessed. The preprocessing includes:

1. Normalizing pixel values from the range 0–255 to 0–1.
2. Reshaping each 28×28 image into a one-dimensional vector containing 784 pixels.

```
# Normalize pixel values (0-255 → 0-1)
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Flatten each 28x28 image into a vector of 784 pixels
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)

print("\nFirst five normalized pixel values:")
print(x_train[0][:5])
```

Training data shape: (60000, 784)

Testing data shape: (10000, 784)

First five normalized pixel values:

[0. 0. 0. 0. 0.]

Step 5: Build the Feed-Forward Neural Network

A Feed-Forward Neural Network (FFNN) consists of an input layer, one or more hidden layers, and an output layer. The network architecture used in this practical is:

- Input Layer: 784 neurons (28×28 flattened image)
- Hidden Layer 1: 128 neurons with ReLU activation
- Hidden Layer 2: 64 neurons with ReLU activation
- Output Layer: 10 neurons with Softmax activation

The ReLU activation function introduces non-linearity, while the Softmax activation function converts the output into probabilities for the 10 digit classes.

```
# Build the Feed-Forward Neural Network
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

# Display the model architecture
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

Step 6: Compile and Train the Model using SGD Optimizer

Before training the neural network, the model must be compiled. The

following components are specified:

- Optimizer: SGD (Stochastic Gradient Descent)
- Loss Function: Sparse Categorical Crossentropy
- Evaluation Metric: Accuracy

The model is then trained for 10 epochs using the training dataset, while 20% of the training data is reserved for validation.

```
# Compile the model using SGD optimizer
model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
history_sgd = model.fit(
    x_train, y_train,
    epochs=10, batch_size=32,
    validation_split=0.2, verbose=1
)
```

```
Epoch 1/10
1500/1500 — 3s 2ms/step - accuracy: 0.8064 - loss: 0.7202 - val_accuracy: 0.9018 - val_loss: 0.3443 Epoch 2/10
1500/1500 — 2s 1ms/step - accuracy: 0.9084 - loss: 0.3228 - val_accuracy: 0.9233 - val_loss: 0.2710
...
Epoch 9/10
1500/1500 — 2s 1ms/step - accuracy: 0.9595 - loss: 0.1409 - val_accuracy: 0.9593 - val_loss: 0.1467 Epoch 10/10
1500/1500 — 2s 1ms/step - accuracy: 0.9625 - loss: 0.1304 - val_accuracy: 0.9602 - val_loss: 0.1391
```

Step 7: Evaluate the Model on the Test Dataset

After training, the model is evaluated using the unseen test dataset. The

evaluation returns:

- Test Loss
- Test Accuracy

These metrics indicate how well the trained model generalizes to new, unseen data.

```
# Evaluate the trained model on the test dataset
test_loss_sgd, test_accuracy_sgd = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(f"SGD Test Loss      : {test_loss_sgd:.4f}")
print(f"SGD Test Accuracy  : {test_accuracy_sgd:.4f}")
```

```
SGD Test Loss      : 0.1338
SGD Test Accuracy : 0.9617
```

Step 8: Create a Reusable Feed-Forward Neural Network

To compare different optimizers fairly, each optimizer must train a **new model** with the **same architecture**.

Instead of writing the same model-building code multiple times, a function named `create_model()` is defined. This function returns a new Feed-Forward Neural Network whenever it is called.

```
# Function to create a new Feed-Forward Neural Network
def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(784,)),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])
    return model
```

```
# Create a fresh model and display its architecture
model = create_model()
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 128)	100,480
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Step 9: Train the Feed-Forward Neural Network using RMSprop Optimizer

The same neural network architecture is trained using the RMSprop optimizer.

RMSprop (Root Mean Square Propagation) is an adaptive learning rate optimization algorithm. It adjusts the learning rate for each parameter individually, which often leads to faster convergence than standard Stochastic Gradient Descent (SGD).

The model is trained for 10 epochs using the same dataset, batch size, and validation split to ensure a fair comparison.

```
# Create a fresh model
model = create_model()

# Compile using RMSprop
model.compile(
    optimizer='rmsprop',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
history_rmsprop = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

Epoch 1/10
1500/1500 ————— 3s 2ms/step - accuracy: 0.9241 - loss: 0.2610 - val_accuracy: 0.9605 - val_loss: 0.1407 Epoch 2/10
1500/1500 ————— 3s 2ms/step - accuracy: 0.9665 - loss: 0.1137 - val_accuracy: 0.9629 - val_loss: 0.1303
...
Epoch 9/10
1500/1500 ————— 3s 2ms/step - accuracy: 0.9918 - loss: 0.0274 - val_accuracy: 0.9747 - val_loss: 0.1326 Epoch 10/10
1500/1500 ————— 3s 2ms/step - accuracy: 0.9929 - loss: 0.0233 - val_accuracy: 0.9744 - val_loss: 0.1276

```
# Evaluate the RMSprop model
test_loss_rmsprop, test_accuracy_rmsprop = model.evaluate(
    x_test,
    y_test,
    verbose=0
)
```

```
)
print(f"RMSprop Test Loss      : {test_loss_rmsprop:.4f}")
print(f"RMSprop Test Accuracy : {test_accuracy_rmsprop:.4f}")
```

```
RMSprop Test Loss      : 0.0859
RMSprop Test Accuracy : 0.9787
```

Step 10: Train the Feed-Forward Neural Network using Adam Optimizer

The same Feed-Forward Neural Network is trained using the Adam optimizer.

Adam (Adaptive Moment Estimation) combines the advantages of Momentum and RMSprop by adapting the learning rate for each parameter while maintaining moving averages of both gradients and squared gradients.

The model is trained using the same architecture and hyperparameters to allow a fair comparison with SGD and RMSprop.

```
# Create a fresh model
model = create_model()

# Compile using Adam optimizer
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
history_adam = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/10
1500/1500 ————— 4s 2ms/step - accuracy: 0.9216 - loss: 0.2699 - val_accuracy: 0.9623 - val_loss: 0.1347 Epoch 2/10
1500/1500 ————— 4s 2ms/step - accuracy: 0.9655 - loss: 0.1129 - val_accuracy: 0.9649 - val_loss: 0.1180
...
Epoch 9/10
1500/1500 ————— 3s 2ms/step - accuracy: 0.9924 - loss: 0.0220 - val_accuracy: 0.9718 - val_loss: 0.1200 Epoch 10/10
1500/1500 ————— 3s 2ms/step - accuracy: 0.9922 - loss: 0.0214 - val_accuracy: 0.9755 - val_loss: 0.1032
```

```
# Evaluate the Adam model
test_loss_adam, test_accuracy_adam = model.evaluate(
    x_test,
    y_test,
    verbose=0
)
print(f"Adam Test Loss      : {test_loss_adam:.4f}")
print(f"Adam Test Accuracy : {test_accuracy_adam:.4f}")
```

```
Adam Test Loss      : 0.0859
Adam Test Accuracy : 0.9787
```

Step 11: Compare the Performance of Different Optimizers

The Feed-Forward Neural Network was trained using three different optimizers:

- SGD
- RMSprop
- Adam

To compare their performance, the test accuracy and test loss obtained from each optimizer are summarized in the following table.

```
# Display comparison table
print("=" * 50)
print("{:<12} {:<15} {:<15}".format("Optimizer", "Test Accuracy", "Test Loss"))
print("=" * 50)

print("{:<12} {:<15.4f} {:<15.4f}".format(
    "SGD",
    test_accuracy_sgd,
    test_loss_sgd
))
```



```
print("{:<12} {:<15.4f} {:<15.4f}".format(
    "RMSprop",
    test_accuracy_rmsprop,
    test_loss_rmsprop
))

print("{:<12} {:<15.4f} {:<15.4f}".format(
    "Adam",
    test_accuracy_adam,
    test_loss_adam
))
```

Optimizer	Test Accuracy	Test Loss
SGD	0.9617	0.1338
RMSprop	0.9787	0.0859
Adam	0.9787	0.0859

Step 12: Plot Training Accuracy Comparison

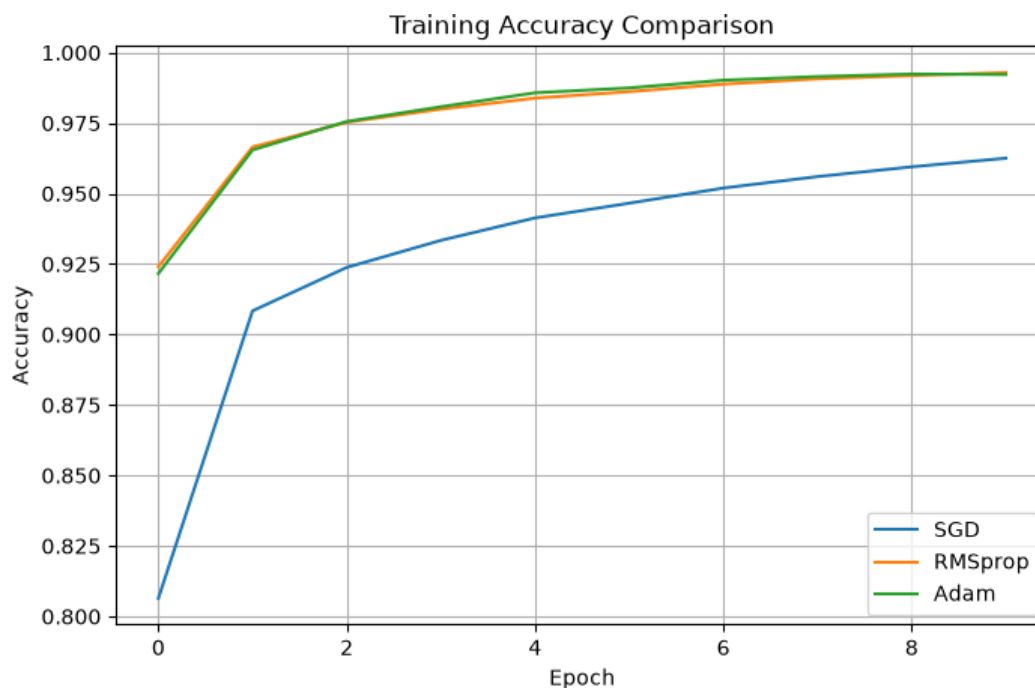
The training accuracy obtained using different optimizers is plotted to visualize their learning performance.

```
plt.figure(figsize=(8,5))

plt.plot(history_sgd.history['accuracy'], label='SGD')
plt.plot(history_rmsprop.history['accuracy'], label='RMSprop')
plt.plot(history_adam.history['accuracy'], label='Adam')

plt.title("Training Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



Step 13: Plot Validation Accuracy Comparison

The validation accuracy obtained using different optimizers is plotted for comparison.

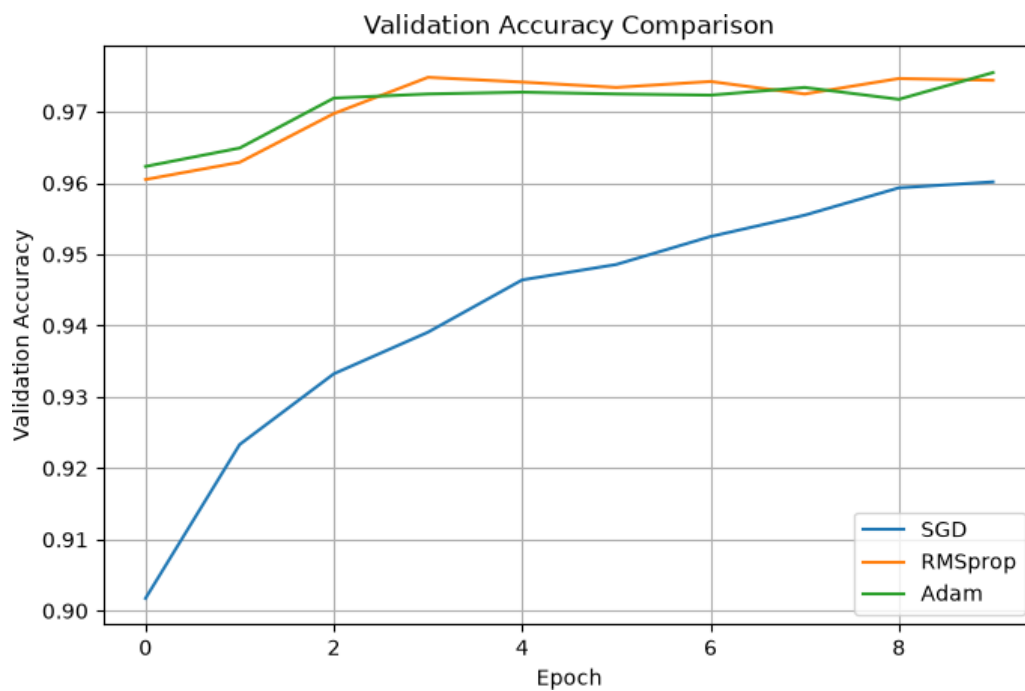
```
plt.figure(figsize=(8,5))

plt.plot(history_sgd.history['val_accuracy'], label='SGD')
plt.plot(history_rmsprop.history['val_accuracy'], label='RMSprop')
plt.plot(history_adam.history['val_accuracy'], label='Adam')

plt.title("Validation Accuracy Comparison")
plt.xlabel("Epoch")
```

```
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



Step 14: Plot Training Loss Comparison

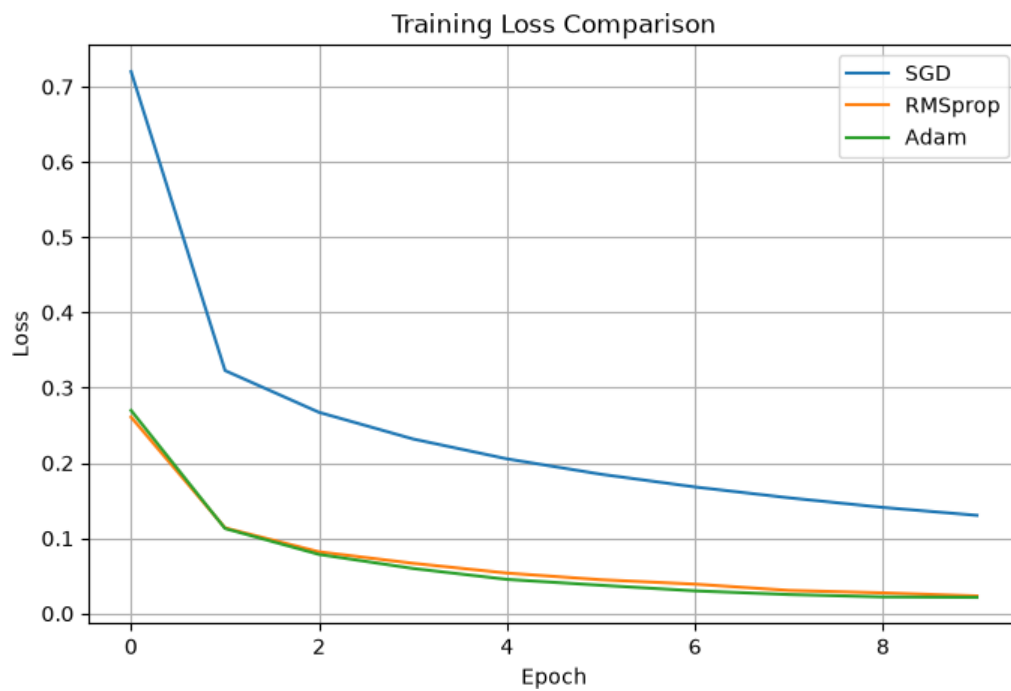
The training loss for each optimizer is plotted to compare the convergence of the models.

```
plt.figure(figsize=(8,5))

plt.plot(history_sgd.history['loss'], label='SGD')
plt.plot(history_rmsprop.history['loss'], label='RMSprop')
plt.plot(history_adam.history['loss'], label='Adam')

plt.title("Training Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```



Result

The Feed-Forward Neural Network was successfully implemented using TensorFlow and trained using three optimization algorithms: SGD, RMSprop, and Adam.

The performance of the optimizers was compared using test accuracy and test loss. It was observed that adaptive optimizers such as RMSprop and Adam converged faster and generally achieved higher accuracy than the SGD optimizer.

Conclusion

The practical demonstrated the implementation of a Feed-Forward Neural Network for handwritten digit classification using the MNIST dataset.

Three optimization algorithms were evaluated under identical conditions. The comparison showed that although SGD successfully trained the network, RMSprop and Adam generally provided faster convergence and better classification performance. Adam proved to be the most effective optimizer for this task due to its adaptive learning rate and efficient parameter updates.

Practical 2: Write a Program to implement regularization to prevent the model from overfitting

Aim

To implement regularization techniques in a Feed-Forward Neural Network using TensorFlow to reduce overfitting and improve the model's generalization performance.

Solution

Step 1: Import Required Libraries

The TensorFlow library is used to build and train the neural network. NumPy is used for numerical operations, and Matplotlib is used to visualize the model's performance.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.18.0
NumPy Version: 1.26.4

Step 2: Load and Preprocess the MNIST Dataset

The MNIST dataset is loaded using TensorFlow. Before training the neural network, the image data is preprocessed by:

1. Normalizing pixel values from the range **0–255** to **0–1**.
2. Flattening each **28 × 28** image into a one-dimensional vector containing **784** features.

These preprocessing steps improve training efficiency and prepare the data for a Feed-Forward Neural Network.

```
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize pixel values
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Flatten the images
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)

# Display dataset information
print("Training data shape :", x_train.shape)
print("Testing data shape :", x_test.shape)
print("Training labels      :", y_train.shape)
print("Testing labels       :", y_test.shape)
```

Training data shape : (60000, 784)
Testing data shape : (10000, 784) Training
labels : (60000,) Testing
labels : (10000,)

Step 3: Build a Feed-Forward Neural Network without Regularization

A baseline Feed-Forward Neural Network is created without using any regularization techniques.

The network contains multiple hidden layers with a large number of neurons. Such models have high learning capacity and may memorize the training data, leading to **overfitting**.

This model will serve as the reference for comparing the effects of regularization techniques in later steps.

```
# Create a baseline model without regularization
baseline_model = tf.keras.Sequential([
```

```
tf.keras.layers.Input(shape=(784,)),
tf.keras.layers.Dense(512, activation='relu'),
tf.keras.layers.Dense(256, activation='relu'),
tf.keras.layers.Dense(128, activation='relu'),
tf.keras.layers.Dense(10, activation='softmax')
])
baseline_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	401,920
dense_1 (Dense)	(None, 256)	131,328
dense_2 (Dense)	(None, 128)	32,896
dense_3 (Dense)	(None, 10)	1,290

Total params: 567,434 (2.16 MB)

Trainable params: 567,434 (2.16 MB)

Non-trainable params: 0 (0.00 B)

Step 4: Compile and Train the Baseline Model

The baseline Feed-Forward Neural Network is compiled using the Adam optimizer and the Sparse Categorical Crossentropy loss function.

The model is trained for **20 epochs** with a batch size of **32**. During training, **20%** of the training dataset is reserved for validation to monitor the model's performance and detect overfitting.

```
# Compile the baseline model
baseline_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the baseline model
history_baseline = baseline_model.fit(
    x_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

Epoch 1/20

1500/1500 — 9s 5ms/step - accuracy: 0.9339 - loss: 0.2181 - val_accuracy: 0.9676 - val_loss: 0.1033 Epoch 2/20

1500/1500 — 8s 5ms/step - accuracy: 0.9707 - loss: 0.0929 - val_accuracy: 0.9633 - val_loss: 0.1221

...

Epoch 18/20

1500/1500 — 12s 8ms/step - accuracy: 0.9957 - loss: 0.0145 - val_accuracy: 0.9762 - val_loss: 0.1518 Epoch 19/20

1500/1500 — 11s 8ms/step - accuracy: 0.9964 - loss: 0.0143 - val_accuracy: 0.9792 - val_loss: 0.1273 Epoch 20/20

1500/1500 — 12s 8ms/step - accuracy: 0.9970 - loss: 0.0110 - val_accuracy: 0.9802 - val_loss: 0.1491

Step 5: Evaluate the Baseline Model

The trained baseline model is evaluated using the unseen MNIST test dataset. The test loss and test accuracy are recorded for later comparison with the regularized model.

```
# Evaluate the baseline model
baseline_loss, baseline_accuracy = baseline_model.evaluate(
    x_test,
    y_test,
    verbose=0
)
print(f"Baseline Test Loss      : {baseline_loss:.4f}")
print(f"Baseline Test Accuracy : {baseline_accuracy:.4f}")
```

Baseline Test Loss : 0.1285

Baseline Test Accuracy : 0.9838

Step 6: Build a Feed-Forward Neural Network with Regularization

To reduce overfitting, two regularization techniques are applied:

1. **L2 Regularization (Weight Decay):** Adds a penalty to large weight values, encouraging simpler models.
2. **Dropout:** Randomly deactivates a percentage of neurons during training, preventing the network from relying too heavily on specific neurons.

The architecture of the network remains similar to the baseline model, allowing a fair comparison of the effect of regularization.

```
# Import regularizer
from tensorflow.keras import regularizers

# Create a model with L2 Regularization and Dropout
regularized_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(
        512,
        activation='relu',
        kernel_regularizer=regularizers.l2(0.001)
    ),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(
        256,
        activation='relu',
        kernel_regularizer=regularizers.l2(0.001)
    ),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(
        128,
        activation='relu',
        kernel_regularizer=regularizers.l2(0.001)
    ),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(10, activation='softmax')
])

regularized_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 512)	401,920
dropout (Dropout)	(None, 512)	0
dense_5 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_6 (Dense)	(None, 128)	32,896
dropout_2 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 10)	1,290

Total params: 567,434 (2.16 MB)

Trainable params: 567,434 (2.16 MB)

Non-trainable params: 0 (0.00 B)

Step 7: Compile and Train the Regularized Model

The regularized Feed-Forward Neural Network is compiled using the Adam optimizer and trained using the same hyperparameters as the baseline model.

Training Parameters:

- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Epochs: 20
- Batch Size: 32
- Validation Split: 20%

Using the same training parameters ensures that any performance differences are due to the regularization techniques rather than changes in the training configuration.

```
# Compile the regularized model
regularized_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the regularized model
history_regularized = regularized_model.fit(
    x_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/20
1500/1500 ————— 10s 6ms/step - accuracy: 0.8955 - loss: 0.8244 - val_accuracy: 0.9546 - val_loss: 0.4622 Epoch 2/20
1500/1500 ————— 9s 6ms/step - accuracy: 0.9402 - loss: 0.4697 - val_accuracy: 0.9590 - val_loss: 0.3715
...
Epoch 19/20
1500/1500 ————— 10s 7ms/step - accuracy: 0.9521 - loss: 0.3396 - val_accuracy: 0.9643 - val_loss: 0.2928 Epoch 20/20
1500/1500 ————— 10s 7ms/step - accuracy: 0.9511 - loss: 0.3408 - val_accuracy: 0.9660 - val_loss: 0.2866
```

Step 8: Evaluate the Regularized Model

After training, the regularized model is evaluated using the unseen MNIST test dataset. The test loss and test accuracy are recorded for comparison with the baseline model.

```
# Evaluate the regularized model
regularized_loss, regularized_accuracy = regularized_model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(f"Regularized Test Loss      : {regularized_loss:.4f}")
print(f"Regularized Test Accuracy : {regularized_accuracy:.4f}")
```

```
Regularized Test Loss      : 0.2892
Regularized Test Accuracy : 0.9646
```

Step 9: Compare the Performance of the Models

The performance of the baseline model and the regularized model is compared using the test accuracy and test loss obtained after evaluation on the MNIST test dataset.

```
# Display comparison table
print("=" * 60)
print("{:<20} {:<18} {:<15}".format("Model", "Test Accuracy", "Test Loss"))
print("=" * 60)

print("{:<20} {:<18.4f} {:<15.4f}".format(
    "Baseline",
    baseline_accuracy,
    baseline_loss
))

print("{:<20} {:<18.4f} {:<15.4f}".format(
    "Regularized",
    regularized_accuracy,
    regularized_loss
))
```

Model	Test Accuracy	Test Loss
Baseline	0.9838	0.1285
Regularized	0.9646	0.2892

Step 10: Training Accuracy Comparison

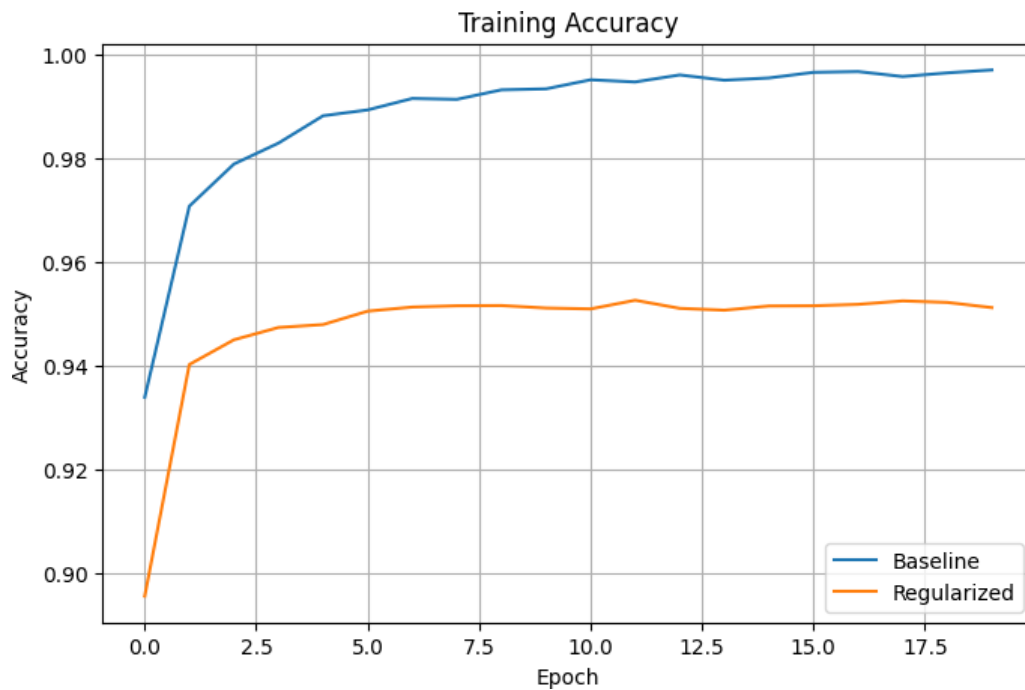
The training accuracy of the baseline model and the regularized model is compared over 20 epochs.

```
plt.figure(figsize=(8,5))

plt.plot(history_baseline.history['accuracy'], label='Baseline')
plt.plot(history_regularized.history['accuracy'], label='Regularized')

plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



Step 11: Validation Accuracy Comparison

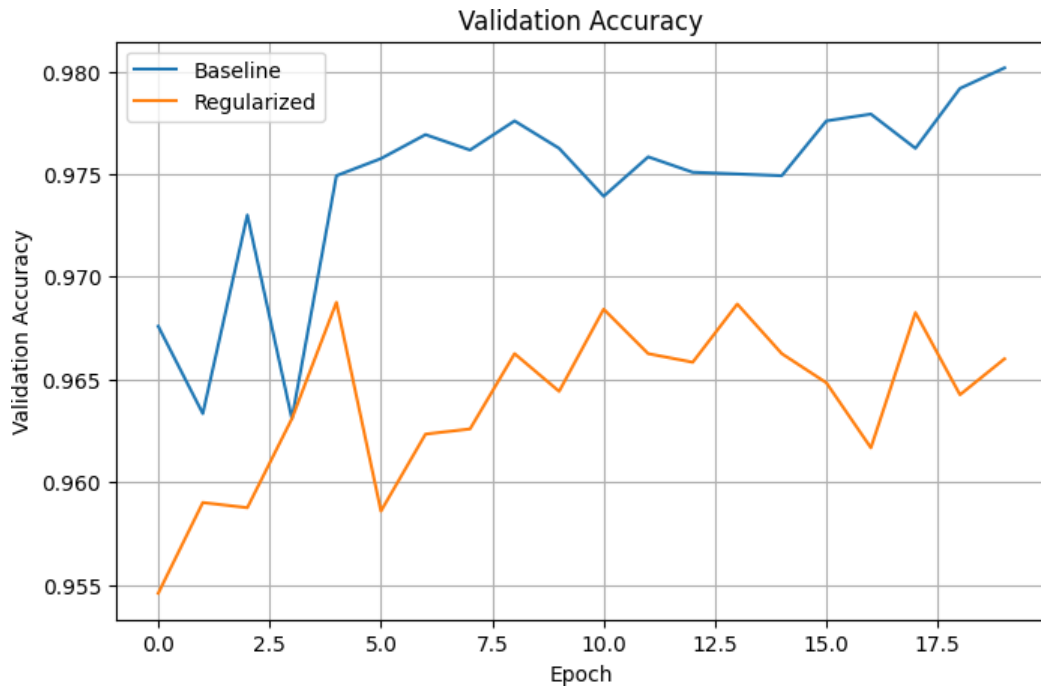
The validation accuracy of both models is compared to observe the effect of regularization on model generalization.

```
plt.figure(figsize=(8,5))

plt.plot(history_baseline.history['val_accuracy'], label='Baseline')
plt.plot(history_regularized.history['val_accuracy'], label='Regularized')

plt.title("Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```

Step 12: Validation Loss Comparison

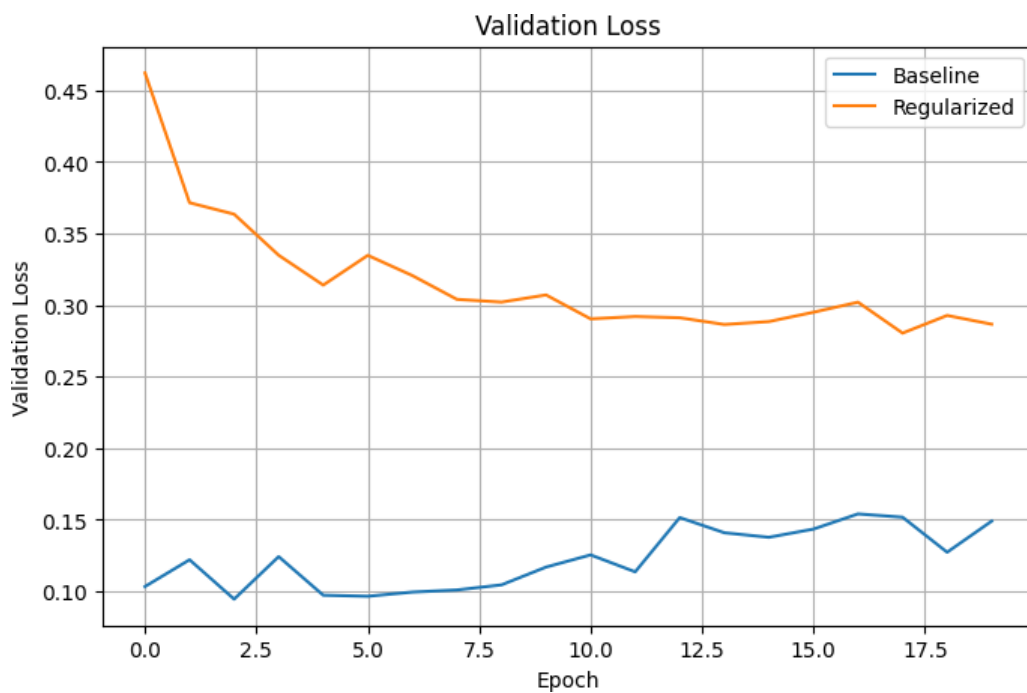
Validation loss is compared to determine whether regularization helps reduce overfitting.

```
plt.figure(figsize=(8,5))

plt.plot(history_baseline.history['val_loss'], label='Baseline')
plt.plot(history_regularized.history['val_loss'], label='Regularized')

plt.title("Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.legend()
plt.grid(True)

plt.show()
```



Step 13: Training Loss Comparison

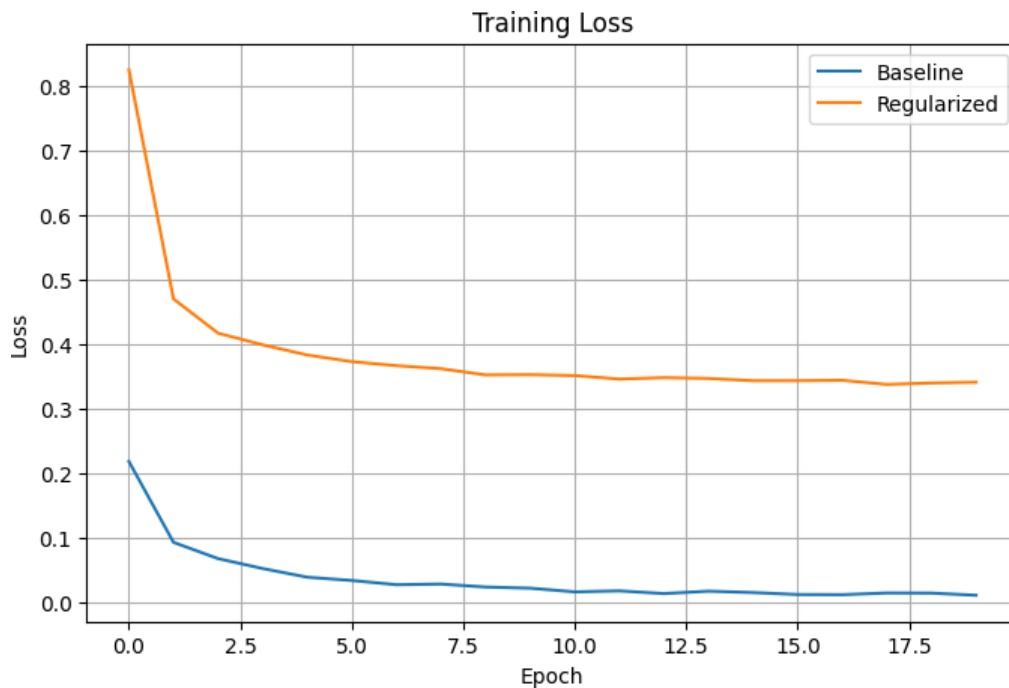
The training loss of the baseline and regularized models is plotted to compare the convergence of the models during training.

```
plt.figure(figsize=(8,5))

plt.plot(history_baseline.history['loss'], label='Baseline')
plt.plot(history_regularized.history['loss'], label='Regularized')

plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```



Result

The Feed-Forward Neural Network was successfully trained with and without regularization techniques.

The regularized model, which used L2 Regularization and Dropout, demonstrated improved generalization performance by reducing overfitting. Although the training accuracy was slightly lower, the validation and test performance remained more stable compared to the baseline model.

Conclusion

In this practical, regularization techniques were successfully implemented in a Feed-Forward Neural Network using TensorFlow.

The baseline model achieved high training accuracy but was more susceptible to overfitting. By applying L2 Regularization and Dropout, the model learned more generalized features and showed improved performance on unseen data.

This practical demonstrates that regularization is an effective technique for improving the robustness and generalization capability of deep learning models.

Practical 3: Implement deep learning for recognizing classes for datasets like CIFAR-10 images for previously unseen images and assign them to one of the 10 classes

Aim

To implement a Convolutional Neural Network (CNN) using TensorFlow for classifying CIFAR-10 images into one of the ten object classes and predict the class of previously unseen images.

Solution

Step 1: Import Required Libraries

The TensorFlow library is used for implementing the Convolutional Neural Network. NumPy is used for numerical computations and Matplotlib is used to visualize the dataset and prediction results.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.18.0
NumPy Version: 1.26.4

Step 2: Load the CIFAR-10 Dataset

The CIFAR-10 dataset is loaded using TensorFlow. It contains 60,000 color images of size 32×32 pixels belonging to 10 different object classes.

The dataset is divided into:

- Training Images: 50,000
- Testing Images: 10,000
- Image Size: $32 \times 32 \times 3$ (RGB)
- Number of Classes: 10

```
# Load the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Display dataset information
print("Training Images Shape :", x_train.shape)
print("Training Labels Shape :", y_train.shape)
print("Testing Images Shape :", x_test.shape)
print("Testing Labels Shape :", y_test.shape)
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 27s
0us/step Training Images Shape : (50000, 32, 32, 3)
Training Labels Shape : (50000, 1)
Testing Images Shape : (10000, 32, 32, 3)
Testing Labels Shape : (10000, 1)

Step 3: Define the Class Labels

The CIFAR-10 dataset contains 10 object classes. A list of class names is created to display meaningful labels instead of numeric values.

```
# Define class names
class_names = [
    "Airplane",
    "Automobile",
    "Bird",
    "Cat",
    "Deer",
    "Dog",
    "Frog",
```

```

"Horse",
"Ship",
"Truck"
]
print(class_names)

```

['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer', 'Dog', 'Frog', 'Horse', 'Ship', 'Truck']

Step 4: Display Sample Images

The first 10 images from the training dataset are displayed along with their corresponding class names. This helps in understanding the type of images present in the CIFAR-10 dataset.

```

# Display the first 10 training images
plt.figure(figsize=(12,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis("off")

plt.tight_layout()
plt.show()

```



Step 5: Preprocess the Dataset

Before training the Convolutional Neural Network (CNN), the image pixel values are normalized.

The original pixel values range from **0 to 255**. They are converted to the range **0 to 1** by dividing each pixel value by **255**. Normalization improves the speed and stability of the learning process.

```

# Normalize the image pixel values
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())

```

Minimum pixel value: 0.0
Maximum pixel value: 1.0

Step 6: Build the Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is constructed for image classification. The architecture consists of:

- Convolution Layers
- Max Pooling Layers
- Flatten Layer
- Fully Connected (Dense) Layers

- Softmax Output Layer

The convolution layers extract important image features, while the pooling layers reduce the image dimensions and computational complexity.

```
# Build the CNN model
cnn_model = tf.keras.Sequential([

    tf.keras.layers.Conv2D(
        filters=32,
        kernel_size=(3,3),
        activation='relu',
        input_shape=(32,32,3)
    ),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(
        filters=64,
        kernel_size=(3,3),
        activation='relu'
    ),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

cnn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 64)	147,520
dense_1 (Dense)	(None, 10)	650

Total params: 167,562 (654.54 KB)
Trainable params: 167,562 (654.54 KB)
Non-trainable params: 0 (0.00 B)

Step 7: Compile and Train the CNN Model

The CNN model is compiled using the Adam optimizer and the Sparse Categorical Crossentropy loss function. The model is trained for 10 epochs with a batch size of 32. During training, 20% of the training dataset is used for validation.

```
# Compile the CNN model
cnn_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
history = cnn_model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

Epoch 1/10
1250/1250 ————— 13s 9ms/step - accuracy: 0.4530 - loss: 1.5130 - val_accuracy: 0.5592 - val_loss: 1.2543 Epoch 2/10
1250/1250 ————— 13s 10ms/step - accuracy: 0.5912 - loss: 1.1608 - val_accuracy: 0.5987 - val_loss: 1.1482
...
Epoch 9/10
1250/1250 ————— 13s 10ms/step - accuracy: 0.7615 - loss: 0.6844 - val_accuracy: 0.6911 - val_loss: 0.9287 Epoch 10/10
1250/1250 ————— 13s 10ms/step - accuracy: 0.7756 - loss: 0.6465 - val_accuracy: 0.6804 - val_loss: 0.9864

Step 8: Evaluate the CNN Model

After training, the CNN model is evaluated using the unseen CIFAR-10 test dataset. The

evaluation returns:

- Test Loss
- Test Accuracy

These values indicate how well the trained model performs on previously unseen images.

```
# Evaluate the CNN model
test_loss, test_accuracy = cnn_model.evaluate(
    x_test,
    y_test,
    verbose=0
)
print(f"Test Loss      : {test_loss:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")
```

```
Test Loss      : 0.9834
Test Accuracy : 0.6838
```

Step 9: Predict Previously Unseen Images

The trained CNN model is used to predict the classes of previously unseen images from the CIFAR-10 test dataset. The

predicted class is compared with the actual class to evaluate the model's performance.

```
# Predict the first 10 unseen test images
predictions = cnn_model.predict(x_test[:10])

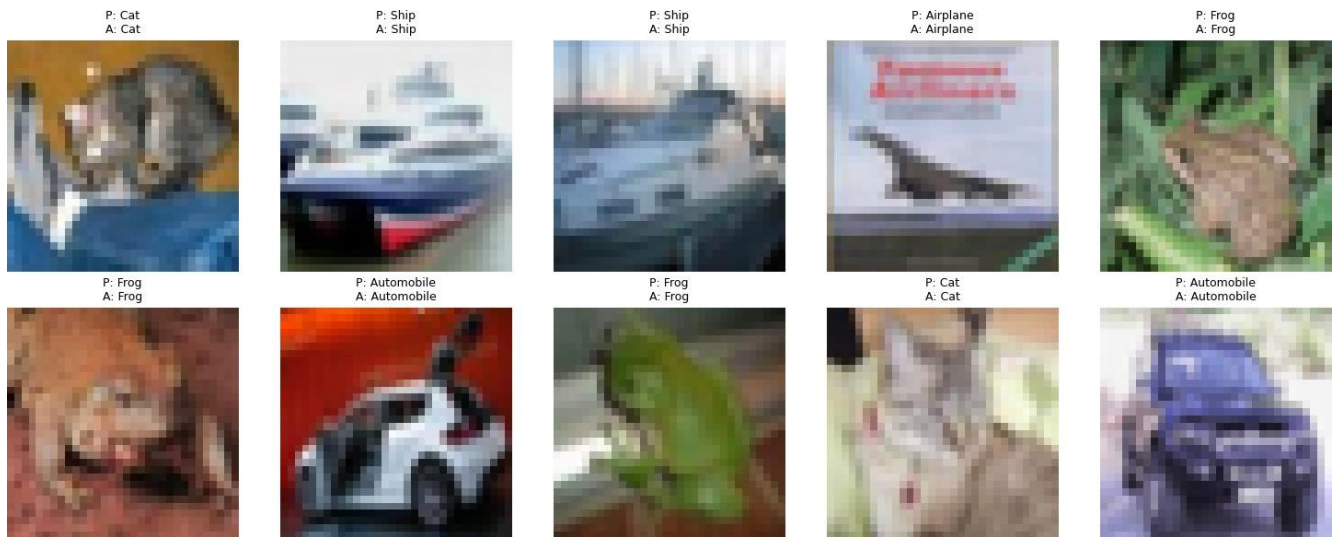
predicted_classes = np.argmax(predictions, axis=1)
actual_classes = y_test[:10].flatten()
```

```
1/1 ————— 0s 73ms/step
```

```
plt.figure(figsize=(15,6))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_test[i])
    predicted = class_names[predicted_classes[i]]
    actual = class_names[actual_classes[i]]
    plt.title(f"P: {predicted}\nA: {actual}", fontsize=9)
    plt.axis("off")

plt.tight_layout()
plt.show()
```



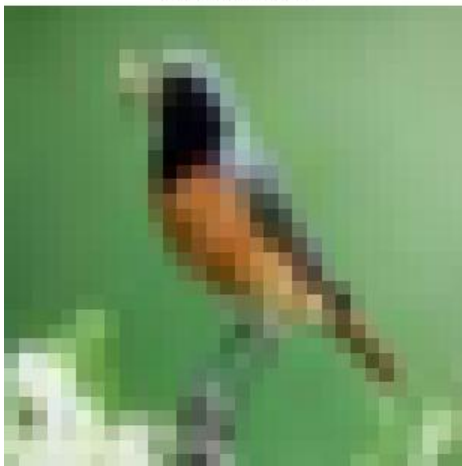
Step 10: Predict the Class of a Single Image

The CNN model predicts the class of a single unseen image from the CIFAR-10 test dataset.

```
# Choose any unseen image
index = 75
prediction = cnn_model.predict(x_test[index:index+1])
predicted_class = np.argmax(prediction)
plt.figure(figsize=(4,4))
plt.imshow(x_test[index])
plt.title(
    f"Predicted: {class_names[predicted_class]}\n"
    f"Actual: {class_names[y_test[index][0]]}"
)
plt.axis("off")
plt.show()
```

1/1 ————— 0s 24ms/step

Predicted: Bird
Actual: Bird



```
# Display prediction probabilities
for i, probability in enumerate(prediction[0]):
    print(f"{class_names[i]:12}: {probability:.4f}")
```

Airplane	:	0.0003
Automobile	:	0.0004
Bird	:	0.0601
Cat	:	0.0795
Deer	:	0.3867
Dog	:	0.2433
Frog	:	0.1565
Horse	:	0.0050
Ship	:	0.0000
Truck	:	0.0683

Result

A Convolutional Neural Network (CNN) was successfully implemented using TensorFlow for image classification on the CIFAR-10 dataset.

The trained model correctly classified most of the previously unseen test images into one of the ten object classes. The model demonstrated good performance in recognizing different categories of color images.

Conclusion

The practical demonstrated the implementation of a Convolutional Neural Network (CNN) for image classification using the CIFAR-10 dataset.

The CNN automatically learned image features through convolution and pooling layers and successfully classified unseen images into their respective classes. The experiment highlights the effectiveness of CNNs in solving computer vision problems and their superiority over traditional Feed-Forward Neural Networks for image-based tasks.

Practical 4: Implement deep learning for the Prediction of the autoencoder from the test data (e.g. MNIST data set)

Aim

To implement a Convolutional Neural Network (CNN) using TensorFlow for classifying CIFAR-10 images into one of the ten object classes and predict the class of previously unseen images.

Solution

Step 1: Import Required Libraries

TensorFlow is used to build the Autoencoder model, NumPy is used for numerical computations, and Matplotlib is used to visualize the reconstructed images.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.18.0
NumPy Version: 1.26.4

Step 2: Load the MNIST Dataset

The MNIST dataset contains grayscale handwritten digit images of size 28×28 pixels. The

dataset consists of:

- 60,000 training images
- 10,000 testing images

For an Autoencoder, the input images are also used as the target output because the objective is to reconstruct the original image.

```
# Load MNIST dataset
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

print("Training Images :", x_train.shape)
print("Testing Images  :", x_test.shape)
```

Training Images : (60000, 28, 28)
Testing Images : (10000, 28, 28)

Step 3: Preprocess the Dataset

Before training the Autoencoder, the image pixel values are normalized to the range 0–1.

Each image is then flattened from a 28×28 matrix into a vector of 784 pixels, which serves as the input to the neural network.

```
# Normalize pixel values
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Flatten the images
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)

print("Training Shape:", x_train.shape)
print("Testing Shape :", x_test.shape)
```

Training Shape: (60000, 784)
Testing Shape : (10000, 784)

Step 4: Build the Autoencoder Model

The Autoencoder consists of two parts:

- **Encoder:** Compresses the input image into a lower-dimensional representation.
- **Decoder:** Reconstructs the original image from the compressed representation.

The input and output layers both contain **784 neurons**, corresponding to the flattened MNIST images.

```
# Define the Encoder
encoder = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu')
])

# Define the Decoder
decoder = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(784, activation='sigmoid')
])

# Combine Encoder and Decoder
autoencoder = tf.keras.Sequential([
    encoder,
    decoder
])

autoencoder.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 32)	110,816
sequential_1 (Sequential)	(None, 784)	111,568

Total params: 222,384 (868.69 KB)

Trainable params: 222,384 (868.69 KB)

Non-trainable params: 0 (0.00 B)

Step 5: Compile and Train the Autoencoder

The Autoencoder is compiled using the Adam optimizer and Mean Squared Error (MSE) loss function.

Unlike image classification, the objective is to minimize the reconstruction error between the original image and the reconstructed image.

```
# Compile the Autoencoder
autoencoder.compile(
    optimizer='adam',
    loss='mse'
)

# Train the Autoencoder
history = autoencoder.fit(
    x_train,
    x_train,
    epochs=20,
    batch_size=256,
    validation_data=(x_test, x_test),
    verbose=1
)
```

Epoch 1/20

235/235 ————— 2s 5ms/step - loss: 0.0625 - val_loss: 0.0348

Epoch 2/20

235/235 ————— 1s 5ms/step - loss: 0.0286 - val_loss: 0.0237

...

Epoch 19/20

235/235 ————— 1s 5ms/step - loss: 0.0102 - val_loss: 0.0099

Epoch 20/20

Step 6: Predict (Reconstruct) Images from the Test Dataset

The trained Autoencoder is used to reconstruct images from the unseen MNIST test dataset.

The reconstructed images are compared with the original images to evaluate the performance of the Autoencoder.

```
# Reconstruct the test images
reconstructed_images = autoencoder.predict(x_test)
```

313/313 ————— 0s 1ms/step

Step 7: Display Original and Reconstructed Images

The original images from the test dataset are displayed alongside their reconstructed versions produced by the Autoencoder. A successful

Autoencoder should generate reconstructed images that closely resemble the original handwritten digits.

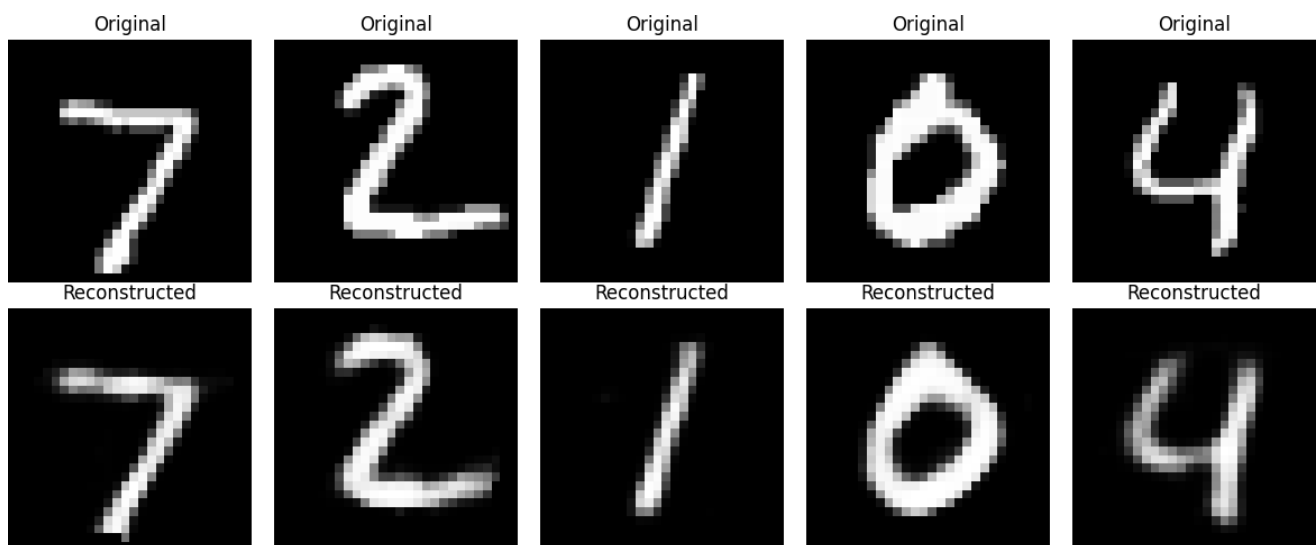
```
# Display original and reconstructed images
n = 5

plt.figure(figsize=(12,5))

for i in range(n):
    # Original Image
    plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")
    plt.axis("off")

    # Reconstructed Image
    plt.subplot(2, n, i + 1 + n)
    plt.imshow(reconstructed_images[i].reshape(28,28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 8: Reconstruction of a Single Test Image

A single image from the MNIST test dataset is reconstructed using the trained Autoencoder and compared with the original image.

```
index = 25

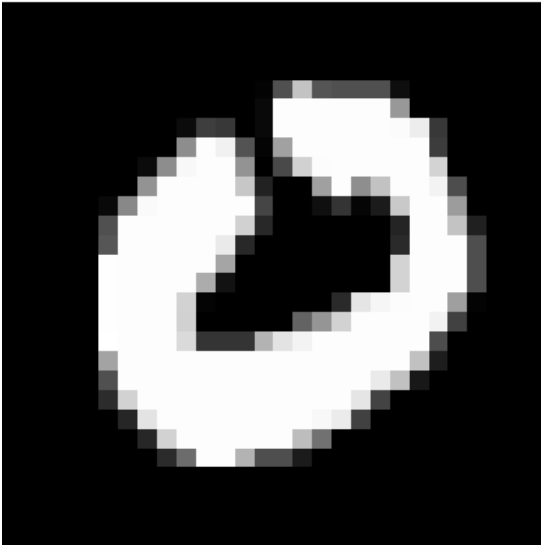
plt.figure(figsize=(8,4))

# Original
plt.subplot(1,2,1)
plt.imshow(x_test[index].reshape(28,28), cmap='gray')
plt.title("Original")
plt.axis("off")

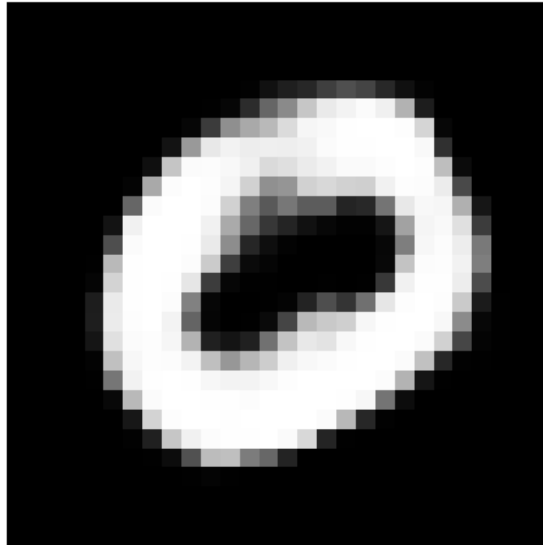
# Reconstructed
plt.subplot(1,2,2)
plt.imshow(reconstructed_images[index].reshape(28,28), cmap='gray')
plt.title("Reconstructed")
plt.axis("off")
```

```
plt.tight_layout()  
plt.show()
```

Original



Reconstructed



Result

The Autoencoder was successfully implemented using TensorFlow and trained on the MNIST dataset.

The trained model reconstructed handwritten digit images from the unseen test dataset with good visual similarity to the original images. This demonstrates that the Autoencoder successfully learned a compressed representation of the input images and reconstructed them effectively.

Conclusion

In this practical, an Autoencoder was implemented for image reconstruction using the MNIST dataset.

The Encoder compressed the input images into a lower-dimensional representation, while the Decoder reconstructed the images from the compressed representation. The reconstructed images closely resembled the original handwritten digits, demonstrating the effectiveness of Autoencoders for feature learning and image reconstruction.

Practical 5: Implement Convolutional Neural Network for Digit Recognition on the MNIST Dataset

Aim

To implement a Convolutional Neural Network (CNN) using TensorFlow for recognizing handwritten digits from the MNIST dataset.

Theory

The MNIST dataset is a standard benchmark dataset for handwritten digit recognition. It contains grayscale images of handwritten digits (0–9), each having a size of 28×28 pixels.

A Convolutional Neural Network (CNN) is a deep learning model specifically designed for image processing tasks. CNNs automatically learn image features such as edges, shapes, and textures using convolutional layers. Pooling layers reduce the dimensions of feature maps, while fully connected layers perform the final classification.

CNNs generally achieve better performance than traditional Feed-Forward Neural Networks because they preserve the spatial information present in images.

Solution

Step 1: Import Required Libraries

TensorFlow is used to implement the Convolutional Neural Network, NumPy is used for numerical operations, and Matplotlib is used for visualizing the handwritten digit images.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.18.0
NumPy Version: 1.26.4

Step 2: Load the MNIST Dataset

The MNIST dataset is loaded using TensorFlow. It consists of 60,000 training images and 10,000 testing images. Each image is a grayscale image of size 28×28 pixels representing handwritten digits from 0 to 9.

```
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Training Images Shape :", x_train.shape)
print("Training Labels Shape :", y_train.shape)
print("Testing Images Shape :", x_test.shape)
print("Testing Labels Shape :", y_test.shape)
```

Training Images Shape : (60000, 28, 28) Training
Labels Shape : (60000,) Testing Images Shape :
(10000, 28, 28) Testing Labels Shape : (10000,)

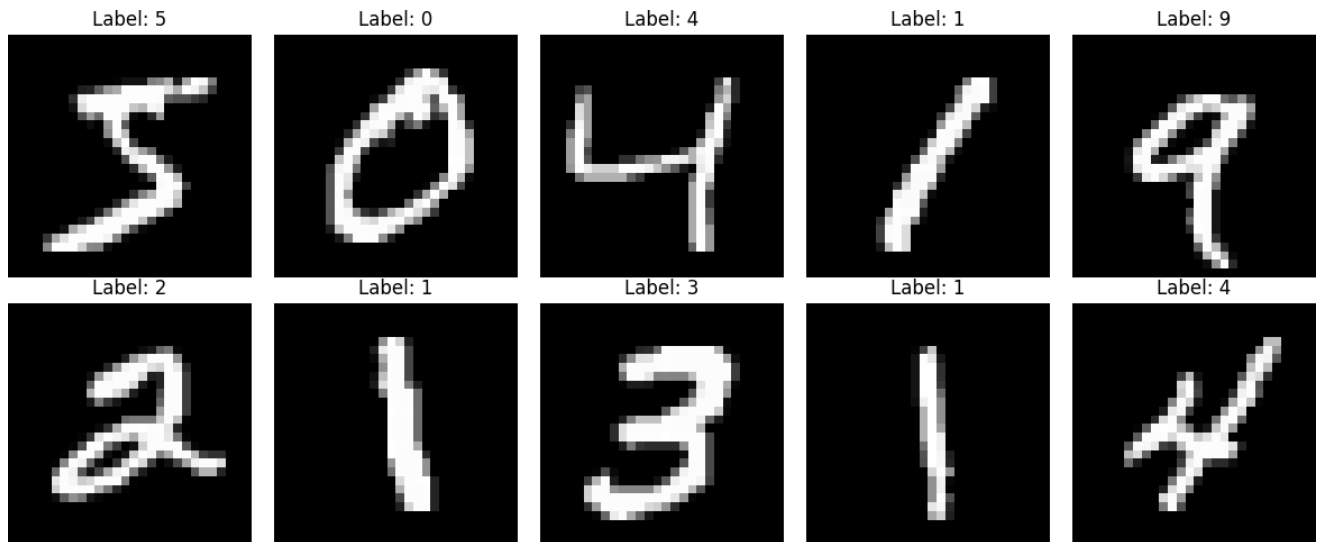
Step 3: Display Sample Images

The first ten images from the MNIST training dataset are displayed along with their corresponding digit labels.

```
plt.figure(figsize=(12,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



Step 4: Preprocess the Dataset

The image pixel values are normalized to the range 0–1. Since the images are grayscale, an additional channel dimension is added, resulting in an input shape of $28 \times 28 \times 1$ for the CNN.

```
# Normalize pixel values
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)
x_test = np.expand_dims(x_test, axis=-1)

print("Training Shape:", x_train.shape)
print("Testing Shape :", x_test.shape)
```

Training Shape: (60000, 28, 28, 1)

Testing Shape : (10000, 28, 28, 1)

Step 5: Build the Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is constructed for handwritten digit recognition. The

architecture consists of:

- Convolution Layer
- Max Pooling Layer
- Second Convolution Layer
- Second Max Pooling Layer
- Flatten Layer
- Fully Connected Dense Layer
- Softmax Output Layer

The convolution layers extract image features, while the pooling layers reduce the feature map dimensions. The dense layers perform the final digit classification.

```
# Build the CNN model

cnn_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(
        filters=32,
        kernel_size=(3,3),
        activation='relu'
    ),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Conv2D(
        filters=64,
        kernel_size=(3,3),
```



```

        activation='relu'
    ),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

cnn_model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

Step 6: Compile and Train the CNN Model

The CNN model is compiled using the Adam optimizer and the Sparse Categorical Crossentropy loss function. The model is trained for 10 epochs with a batch size of 32. During training, 20% of the training dataset is reserved for validation.

```

# Compile the CNN model
cnn_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the CNN model
history = cnn_model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

```

Epoch 1/10

1500/1500 ————— 11s 7ms/step - accuracy: 0.9532 - loss: 0.1526 - val_accuracy: 0.9789 - val_loss: 0.0663 Epoch 2/10

1500/1500 ————— 11s 7ms/step - accuracy: 0.9848 - loss: 0.0478 - val_accuracy: 0.9881 - val_loss: 0.0381

...

Epoch 9/10

1500/1500 ————— 11s 7ms/step - accuracy: 0.9972 - loss: 0.0089 - val_accuracy: 0.9897 - val_loss: 0.0472 Epoch 10/10

1500/1500 ————— 13s 9ms/step - accuracy: 0.9976 - loss: 0.0075 - val_accuracy: 0.9911 - val_loss: 0.0390

Step 7: Evaluate the CNN Model

The trained CNN model is evaluated using the unseen MNIST test dataset. The test loss and test accuracy are calculated to measure the model's performance.

```

# Evaluate the CNN model
test_loss, test_accuracy = cnn_model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(f"Test Loss      : {test_loss:.4f}")
print(f"Test Accuracy  : {test_accuracy:.4f}")

```

Test Loss :0.0395
Test Accuracy : 0.9898

Step 8: Predict Handwritten Digits from the Test Dataset

The trained CNN model is used to predict handwritten digits from the unseen MNIST test dataset. The predicted digit is compared with the actual digit to evaluate the model's performance.

```
# Predict the first 10 test images
predictions = cnn_model.predict(x_test[:10])

predicted_labels = np.argmax(predictions, axis=1)
actual_labels = y_test[:10]
```

1/1 ————— 0s 58ms/step

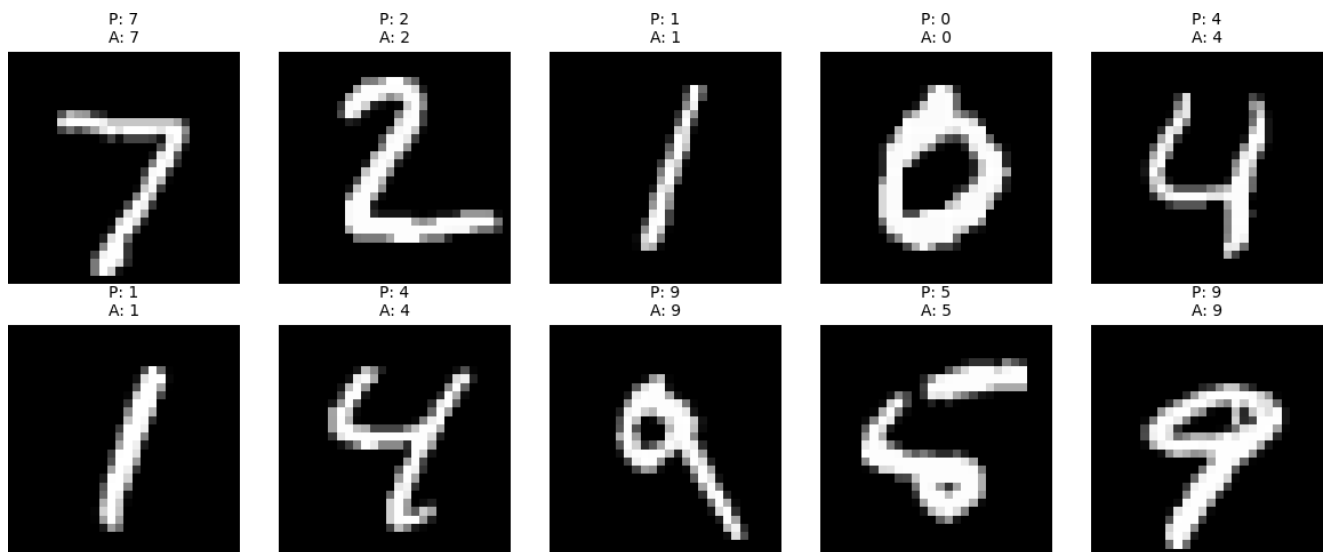
Step 9: Display Prediction Results

The first ten handwritten digit images from the test dataset are displayed along with their predicted and actual digit labels.

```
plt.figure(figsize=(12,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title(
        f"P: {predicted_labels[i]}\nA: {actual_labels[i]}",
        fontsize=10
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```



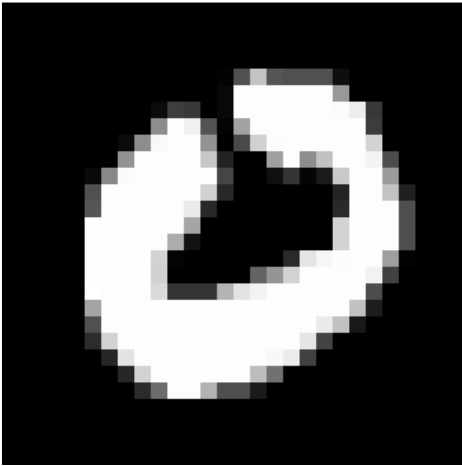
Step 10: Predict a Single Handwritten Digit

A single handwritten digit from the MNIST test dataset is selected and classified using the trained CNN model.

```
# Select a test image
index = 25
prediction = cnn_model.predict(x_test[index:index+1])
predicted_digit = np.argmax(prediction)
plt.figure(figsize=(4,4))
plt.imshow(x_test[index].reshape(28,28), cmap='gray')
plt.title(
    f"Predicted: {predicted_digit}\nActual: {y_test[index]}"
)
plt.axis("off")
plt.show()
```

1/1 ————— 0s 63ms/step

Predicted: 0
Actual: 0



Step 11: Training and Validation Accuracy

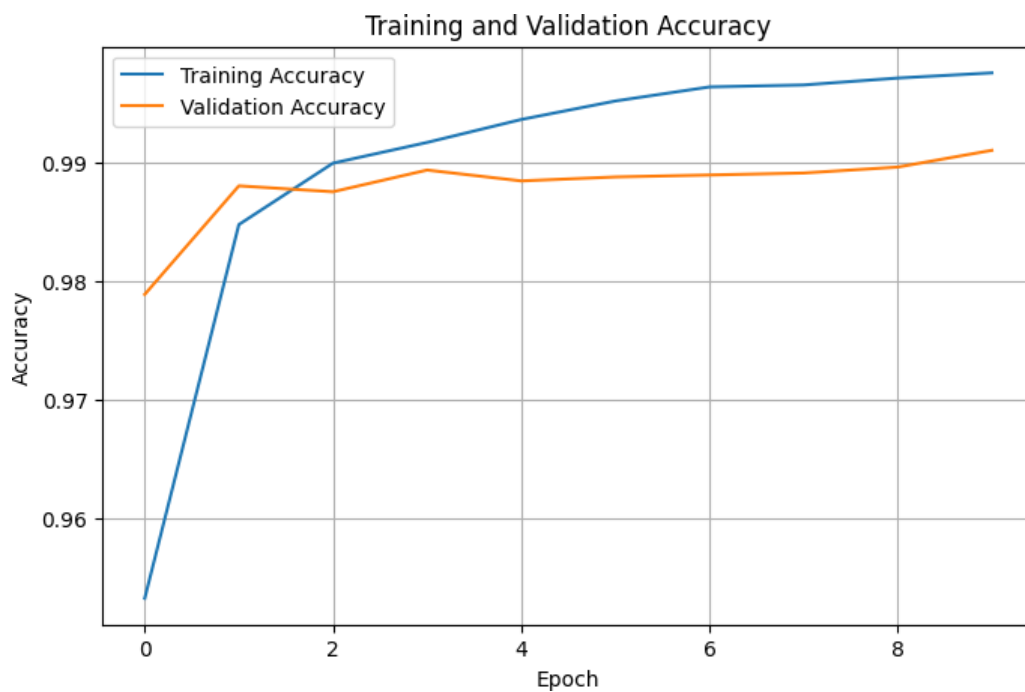
The training and validation accuracy are plotted to observe the learning performance of the CNN model during training.

```
plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()
```



Step 12: Training and Validation Loss

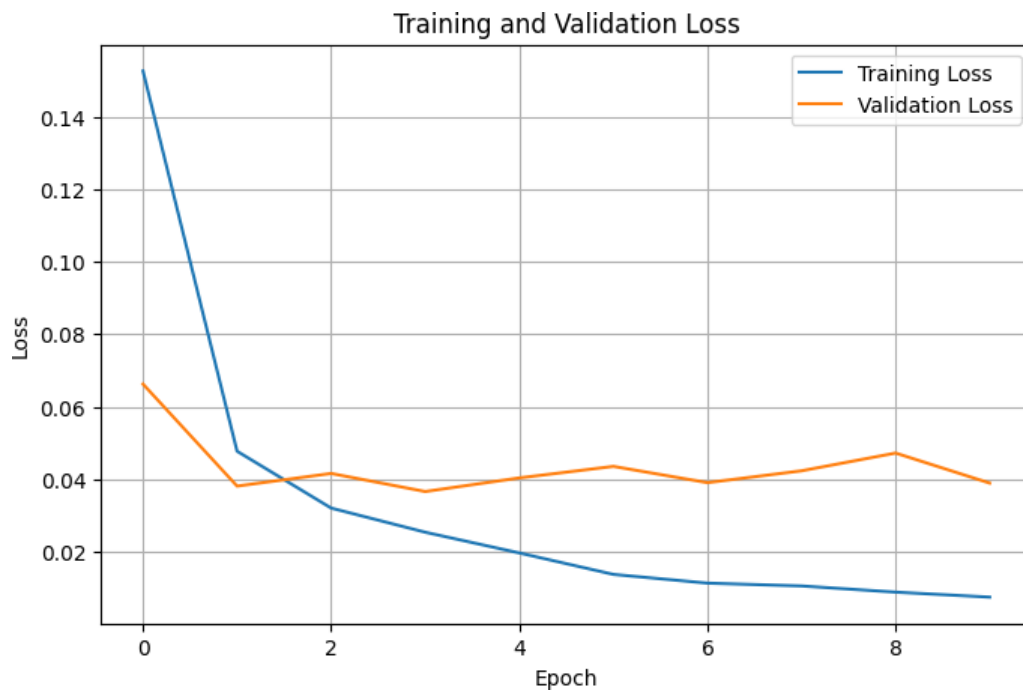
The training and validation loss are plotted to evaluate the convergence of the CNN model during training.

```
plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```



Result

A Convolutional Neural Network (CNN) was successfully implemented using TensorFlow for handwritten digit recognition on the MNIST dataset.

The trained model accurately classified handwritten digits from the unseen test dataset with high test accuracy. The prediction results demonstrate the effectiveness of CNNs in image recognition tasks.

Conclusion

In this practical, a Convolutional Neural Network (CNN) was developed for handwritten digit recognition using the MNIST dataset.

The CNN successfully learned image features through convolution and pooling operations and achieved high accuracy in recognizing unseen handwritten digits. This practical demonstrates the effectiveness of CNNs for image classification problems.

Practical 6: Write a program to implement Transfer Learning on the suitable dataset

Aim

To implement Transfer Learning using the pre-trained MobileNetV2 model in TensorFlow for image classification on the Flower Photos dataset.

Theory

Transfer Learning is a deep learning technique in which a model that has already been trained on a large dataset is reused for a new but related task.

Instead of training an entire neural network from scratch, a pre-trained model is used as a feature extractor. Only the final classification layers are trained on the new dataset.

In this practical, the MobileNetV2 model pre-trained on the ImageNet dataset is used. The Flower Photos dataset contains five flower categories:

- Daisy
- Dandelion
- Roses
- Sunflowers
- Tulips

Transfer Learning significantly reduces training time while providing high classification accuracy.

Solution

Step 1: Import Required Libraries

The required Python libraries are imported for dataset handling, image processing, visualization, and implementing the Transfer Learning model.

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

import pathlib
import os

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.18.0
NumPy Version: 1.26.4

Step 2: Download the Flower Photos Dataset

The Flower Photos dataset is downloaded from the official TensorFlow repository. A

datasets directory is created inside the current project using a relative path.

```
# Create a datasets folder inside the current project
project_dir = pathlib.Path(os.getcwd())
datasets_dir = project_dir / "datasets"
datasets_dir.mkdir(exist_ok=True)

# Download the dataset into the datasets folder
dataset_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"

dataset_path = tf.keras.utils.get_file(
    fname="flower_photos",
    origin=dataset_url,
    untar=True,
```

```
cache_dir=str(datasets_dir),
cache_subdir=""
)

dataset_dir = pathlib.Path(dataset_path)
```

Step 3: Explore the Dataset

The downloaded dataset directory is explored to determine the total number of images available for training the model.

```
if (dataset_dir / "flower_photos").exists():
    dataset_dir = dataset_dir / "flower_photos"

image_count = len(list(dataset_dir.glob("*/*.jpg")))

print("Total Images:", image_count)
```

Total Images: 3670

```
class_names = sorted(
    [item.name for item in dataset_dir.iterdir() if item.is_dir()]
)
print(class_names)
```

['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']

Step 4: Load the Dataset

The Flower Photos dataset is loaded from the project directory using TensorFlow's `image_dataset_from_directory()` function.

The dataset is automatically divided into training and validation sets using an 80:20 split. Each image is resized to **224 × 224 pixels**, which is the required input size for the MobileNetV2 model.

```
# Parameters
batch_size = 32
img_height = 224
img_width = 224

# Training dataset
train_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size
)

# Validation dataset
val_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size
)
```

Found 3670 files belonging to 5 classes. Using 2936 files for training.

Found 3670 files belonging to 5 classes. Using 734 files for validation.

Step 5: Display the Class Names

The class names detected from the dataset are displayed.

```
print("Classes:")
for class_name in class_names:
    print(class_name)
```

Classes: daisy
dandelion roses
sunflowers tulips

Step 6: Optimize Dataset Performance

Dataset prefetching is enabled to improve training performance by overlapping data preprocessing and model execution.

```
AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)

print("Dataset optimization completed.")
```

Dataset optimization completed.

Step 7: Display Sample Images

A few sample flower images from the training dataset are displayed along with their corresponding class labels.

```
plt.figure(figsize=(12, 12))

for images, labels in train_ds.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i].numpy()])
        plt.axis("off")

plt.tight_layout()
plt.show()
```

dandelion



roses



sunflowers



sunflowers



dandelion



roses



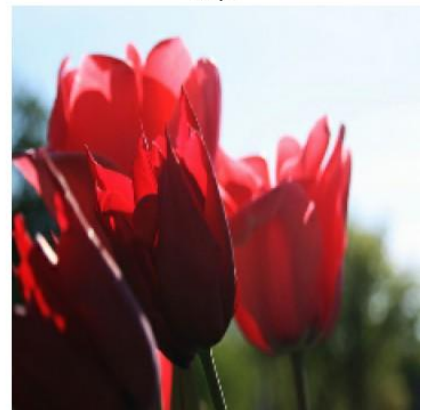
tulips



tulips



tulips



Step 8: Load the Pre-trained MobileNetV2 Model

The MobileNetV2 model pre-trained on the ImageNet dataset is loaded without its original classification layer (`include_top=False`). The pre-trained model acts as a feature extractor, while its weights are frozen so that only the newly added classification layers are trained.

```
# Load the pre-trained MobileNetV2 model
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)

# Freeze the base model
base_model.trainable = False

print("Base model loaded successfully.")
print("Trainable:", base_model.trainable)
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
 9406464/9406464 —————
 1s 0us/step Base model loaded successfully.

Trainable: False

Step 9: Build the Transfer Learning Model

A custom classification head is added on top of the pre-trained MobileNetV2 model. The

model consists of:

- MobileNetV2 (Feature Extractor)
- Global Average Pooling Layer
- Dropout Layer
- Dense Output Layer with Softmax Activation

The output layer contains five neurons corresponding to the five flower classes.

```
# Build the Transfer Learning model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(224, 224, 3)),
    tf.keras.layers.Rescaling(1./127.5, offset=-1),
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(len(class_names), activation="softmax")
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_1 (Dropout)	(None, 1280)	0
dense_1 (Dense)	(None, 5)	6,405

Total params: 2,264,389 (8.64 MB)

Trainable params: 6,405 (25.02 KB)

Non-trainable params: 2,257,984 (8.61 MB)

Step 10: Compile and Train the Model

The Transfer Learning model is compiled using the Adam optimizer and Sparse Categorical Crossentropy loss function. The model is trained using the Flower Photos training dataset and validated using the validation dataset.

```
# Compile the model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Train the model
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5,
    verbose=1
)
```

```
Epoch 1/5
92/92 — 47s 480ms/step - accuracy: 0.6727 - loss: 0.8659 - val_accuracy: 0.8529 - val_loss: 0.4451 Epoch 2/5
92/92 — 45s 493ms/step - accuracy: 0.8495 - loss: 0.4218 - val_accuracy: 0.8747 - val_loss: 0.3589 Epoch 3/5
92/92 — 45s 493ms/step - accuracy: 0.8835 - loss: 0.3370 - val_accuracy: 0.8883 - val_loss: 0.3231 Epoch 4/5
92/92 — 46s 495ms/step - accuracy: 0.9005 - loss: 0.2904 - val_accuracy: 0.8978 - val_loss: 0.3036 Epoch 5/5
92/92 — 60s 652ms/step - accuracy: 0.9149 - loss: 0.2610 - val_accuracy: 0.9019 - val_loss: 0.2961
```

Step 11: Evaluate the Transfer Learning Model

The trained Transfer Learning model is evaluated using the validation dataset. The validation loss and validation accuracy indicate how well the model classifies unseen flower images.

```
# Evaluate the model
loss, accuracy = model.evaluate(val_ds, verbose=1)

print(f"\nValidation Loss      : {loss:.4f}")
print(f"Validation Accuracy : {accuracy:.4f}")
```

```
0.2961 Validation Loss      : 0.2961
Validation Accuracy : 0.9019
```

Step 12: Predict Flower Classes

The trained Transfer Learning model predicts the flower class of unseen validation images. The predicted class is compared with the actual class label.

```
# Get one batch from the validation dataset
images, labels = next(iter(val_ds))
predictions = model.predict(images)
predicted_labels = np.argmax(predictions, axis=1)
```

1/1 ————— 1s 1s/step

Step 13: Display Prediction Results

The predicted flower class is compared with the actual class for a sample of validation images.

```
plt.figure(figsize=(12, 12))

for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.imshow(images[i].numpy().astype("uint8"))
    predicted = class_names[predicted_labels[i]]
    actual = class_names[labels[i].numpy()]
    color = "green" if predicted == actual else "red"
    plt.title(
        f"P: {predicted}\nA: {actual}",
        color=color,
        fontsize=10
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```

P: sunflowers
A: sunflowers



P: tulips
A: tulips



P: roses
A: roses



P: dandelion
A: dandelion



P: roses
A: roses



P: daisy
A: roses



P: dandelion
A: dandelion



P: daisy
A: daisy



P: dandelion
A: dandelion



Step 14: Training and Validation Accuracy

The graph below shows the training and validation accuracy during the training process.

```
plt.figure(figsize=(8,5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

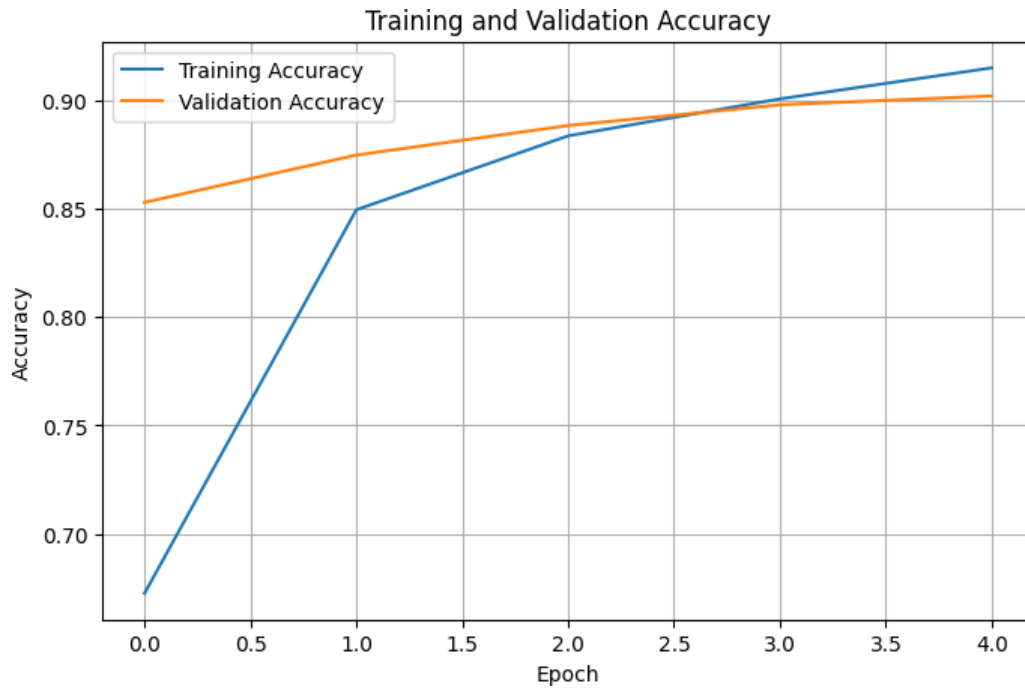
plt.title("Training and Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()

plt.grid(True)

plt.show()
```



Step 15: Training and Validation Loss

The graph below shows the training and validation loss during the training process.

```
plt.figure(figsize=(8,5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

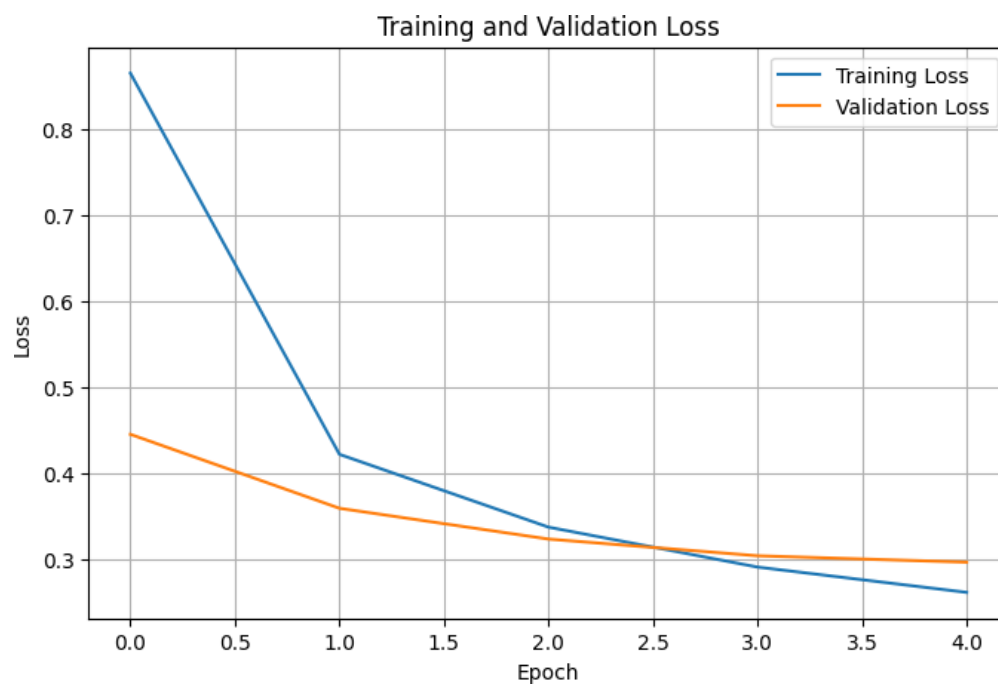
plt.title("Training and Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()

plt.grid(True)

plt.show()
```



Result

Transfer Learning was successfully implemented using the pre-trained MobileNetV2 model on the Flower Photos dataset.

The model accurately classified flower images into five categories: Daisy, Dandelion, Roses, Sunflowers, and Tulips. The pre-trained feature extractor significantly reduced training time while achieving good validation accuracy on previously unseen images.

Conclusion

In this practical, Transfer Learning was implemented using the MobileNetV2 model pre-trained on the ImageNet dataset. The pre-trained model was used as a feature extractor, and a custom classification layer was trained on the Flower Photos dataset.

The model successfully classified unseen flower images with good accuracy, demonstrating the effectiveness of Transfer Learning for image classification tasks while reducing both training time and computational requirements.

Practical 7: Write a program for the Implementation of a Generative Adversarial Network for generating synthetic shapes

Aim

To implement a Generative Adversarial Network (GAN) using TensorFlow for generating synthetic handwritten digit images based on the MNIST dataset.

Theory

A Generative Adversarial Network (GAN) is a deep learning model consisting of two neural networks:

1. **Generator**
2. **Discriminator**

The Generator creates synthetic images from random noise, while the Discriminator attempts to distinguish between real and generated images.

During training, both networks compete with each other:

- The Generator learns to produce realistic images.
- The Discriminator learns to identify fake images.

This adversarial training enables the Generator to create highly realistic synthetic images over time. GANs are widely used for image generation, image enhancement, style transfer, and data augmentation.

Solution

Step 1: Import Required Libraries

The required libraries are imported for implementing the Generative Adversarial Network using TensorFlow.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.18.0
NumPy Version: 1.26.4

Step 2: Load the MNIST Dataset

The MNIST dataset is loaded using TensorFlow. The dataset contains grayscale handwritten digit images of size 28×28 pixels. Only the training images are required because the GAN learns to generate images from the training distribution.

```
# Load MNIST dataset
(x_train, _), ( , ) = tf.keras.datasets.mnist.load_data()

print("Training Images:", x_train.shape)
```

Training Images: (60000, 28, 28)

Step 3: Preprocess the Dataset

The image pixel values are normalized from the range 0–255 to the range -1 to 1.

An additional channel dimension is added because the Generator and Discriminator operate on image tensors of shape $28 \times 28 \times 1$.

```
# Normalize images
x_train = x_train.astype("float32")
x_train = (x_train - 127.5) / 127.5
```

```
# Add channel dimension
x_train = np.expand_dims(x_train, axis=-1)

print("Dataset Shape:", x_train.shape)
```

Dataset Shape: (60000, 28, 28, 1)

Step 4: Create the Training Dataset

The training images are converted into TensorFlow batches to improve training efficiency.

```
BUFFER_SIZE = 60000
BATCH_SIZE = 256

train_dataset = (
    tf.data.Dataset
        .from_tensor_slices(x_train)
        .shuffle(BUFFER_SIZE)
        .batch(BATCH_SIZE)
)

print("Training dataset created successfully.")
```

Training dataset created successfully.

Step 5: Build the Generator Network

The Generator receives a random noise vector of size 100 as input and generates a synthetic handwritten digit image of size $28 \times 28 \times 1$.

The Generator uses Dense, Batch Normalization, LeakyReLU, Reshape, and Conv2DTranspose layers to progressively generate realistic images.

```
def build_generator():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(100,)),
        tf.keras.layers.Dense(7 * 7 * 256, use_bias=False),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.LeakyReLU(),
        tf.keras.layers.Reshape((7, 7, 256)),
        tf.keras.layers.Conv2DTranspose(
            128,
            (5,5),
            strides=(1,1),
            padding="same",
            use_bias=False
        ),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.LeakyReLU(),
        tf.keras.layers.Conv2DTranspose(
            64,
            (5,5),
            strides=(2,2),
            padding="same",
            use_bias=False
        ),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.LeakyReLU(),
        tf.keras.layers.Conv2DTranspose(
            1,
            (5,5),
            strides=(2,2),
            padding="same",
            use_bias=False,
            activation="tanh"
        )
    ])

    return model
```

```
generator = build_generator()

generator.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 12544)	1,254,400
batch_normalization (BatchNormalization)	(None, 12544)	50,176
leaky_re_lu (LeakyReLU)	(None, 12544)	0
reshape (Reshape)	(None, 7, 7, 256)	0
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 128)	819,200
batch_normalization_1 (BatchNormalization)	(None, 7, 7, 128)	512
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 64)	204,800
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_2 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	1,600

Total params: 2,330,944 (8.89 MB)

Trainable params: 2,305,472 (8.79 MB)

Non-trainable params: 25,472 (99.50 KB)

Step 6: Generate a Sample Image

A random noise vector is passed through the Generator to produce an initial synthetic image before training.

```
noise = tf.random.normal([1, 100])

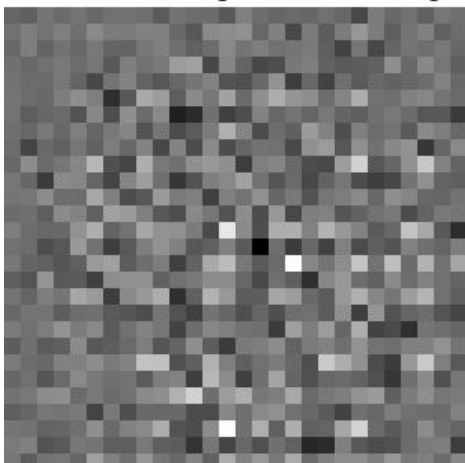
generated_image = generator(noise, training=False)

print("Generated Image Shape:", generated_image.shape)
```

Generated Image Shape: (1, 28, 28, 1)

```
plt.figure(figsize=(4,4))
plt.imshow(
    generated_image[0, :, :, 0],
    cmap="gray"
)
plt.title("Generated Image Before Training")
plt.axis("off")
plt.show()
```

Generated Image Before Training



Step 7: Build the Discriminator Network

The Discriminator receives an image as input and predicts whether it is a real handwritten digit from the MNIST dataset or a fake image generated by the Generator.

The network uses convolution layers followed by a Dense output layer.

```
def build_discriminator():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28,28,1)),
        tf.keras.layers.Conv2D(
            64,
            (5,5),
            strides=(2,2),
            padding="same"
        ),
        tf.keras.layers.LeakyReLU(),
        tf.keras.layers.Dropout(0.3),
        tf.keras.layers.Conv2D(
            128,
            (5,5),
            strides=(2,2),
            padding="same"
        ),
        tf.keras.layers.LeakyReLU(),
        tf.keras.layers.Dropout(0.3),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(1)
    ])

    return model
```

```
discriminator = build_discriminator()

discriminator.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1,664
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 1)	6,273

Total params: 212,865 (831.50 KB)
Trainable params: 212,865 (831.50 KB)
Non-trainable params: 0 (0.00 B)

Step 8: Test the Discriminator

The generated image is passed to the Discriminator before training. Since the model has not yet been trained, its output does not have any meaningful interpretation.

```
decision = discriminator(generated_image)

print("Discriminator Output:")

print(decision)
```

Discriminator Output:
tf.Tensor([[0.00210197]], shape=(1, 1), dtype=float32)

Step 9: Define the Loss Functions

The GAN uses Binary Crossentropy as the loss function.

- The **Generator** attempts to fool the Discriminator into classifying fake images as real.
- The **Discriminator** attempts to correctly classify real and generated images.

```
# Binary Crossentropy Loss
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)
```

```
# Generator Loss
def generator_loss(fake_output):
    return cross_entropy(
        tf.ones_like(fake_output),
        fake_output
    )

# Discriminator Loss
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(
        tf.ones_like(real_output),
        real_output
    )
    fake_loss = cross_entropy(
        tf.zeros_like(fake_output),
        fake_output
    )
    total_loss = real_loss + fake_loss

    return total_loss
```

Step 10: Define the Optimizers

The Adam optimizer is used to update the parameters of both the Generator and the Discriminator during training.

```
generator_optimizer = tf.keras.optimizers.Adam(1e-4)

discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

Step 11: Define the GAN Training Step

The Generator and Discriminator are trained together in each training step using TensorFlow's GradientTape for automatic differentiation.

```
noise_dim = 100

@tf.function
def train_step(images):
    noise = tf.random.normal([tf.shape(images)[0], noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)
        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)
        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(
        gen_loss,
        generator.trainable_variables
    )

    gradients_of_discriminator = disc_tape.gradient(
        disc_loss,
        discriminator.trainable_variables
    )

    generator_optimizer.apply_gradients(
        zip(
            gradients_of_generator,
            generator.trainable_variables
        )
    )

    discriminator_optimizer.apply_gradients(
        zip(
            gradients_of_discriminator,
            discriminator.trainable_variables
        )
    )
```

```
return gen_loss, disc_loss
```

Step 12: Train the GAN

The Generator and Discriminator are trained simultaneously for multiple epochs. During each epoch, the Generator creates synthetic images while the Discriminator learns to distinguish between real and generated images.

```
EPOCHS = 20

for epoch in range(EPOCHS):
    for image_batch in train_dataset:
        gen_loss, disc_loss = train_step(image_batch)

    print(
        f"Epoch {epoch + 1}/{EPOCHS} | "
        f"Generator Loss: {gen_loss:.4f} | "
        f"Discriminator Loss: {disc_loss:.4f}"
    )
```

Epoch 1/20 | Generator Loss: 0.9276 | Discriminator Loss: 1.0700 Epoch 2/20 |
Generator Loss: 0.6313 | Discriminator Loss: 1.5803 Epoch 3/20 | Generator Loss:
0.8905 | Discriminator Loss: 1.0800 Epoch 4/20 | Generator Loss: 1.1883 |
Discriminator Loss: 1.1086 Epoch 5/20 | Generator Loss: 0.8515 | Discriminator Loss:
1.1639 Epoch 6/20 | Generator Loss: 1.0860 | Discriminator Loss: 0.9226 Epoch 7/20 |
Generator Loss: 0.8433 | Discriminator Loss: 1.6870 Epoch 8/20 | Generator Loss:
0.8240 | Discriminator Loss: 1.4004 Epoch 9/20 | Generator Loss: 0.9708 |
Discriminator Loss: 1.1354

Epoch	10/20		Generator	Loss:	0.8750		Discriminator	Loss:	1.2263
Epoch	11/20		Generator	Loss:	0.7673		Discriminator	Loss:	1.3076
Epoch	12/20		Generator	Loss:	0.7920		Discriminator	Loss:	1.4610
Epoch	13/20		Generator	Loss:	0.9945		Discriminator	Loss:	1.1038
Epoch	14/20		Generator	Loss:	0.8924		Discriminator	Loss:	1.2635
Epoch	15/20		Generator	Loss:	1.3223		Discriminator	Loss:	0.8096
Epoch	16/20		Generator	Loss:	1.1657		Discriminator	Loss:	1.0632
Epoch	17/20		Generator	Loss:	1.1509		Discriminator	Loss:	1.0350
Epoch	18/20		Generator	Loss:	1.2309		Discriminator	Loss:	0.8126
Epoch	19/20		Generator	Loss:	1.1553		Discriminator	Loss:	1.1358
Epoch	20/20		Generator	Loss:	1.2680		Discriminator	Loss:	1.0504

Step 13: Generate Synthetic Handwritten Digits

After training, the Generator is used to create synthetic handwritten digit images from random noise vectors. Each generated image represents a unique handwritten digit learned from the MNIST training dataset.

```
# Generate 16 random noise vectors
num_examples = 16
noise = tf.random.normal([num_examples, noise_dim])

# Generate synthetic images
generated_images = generator(noise, training=False)
```

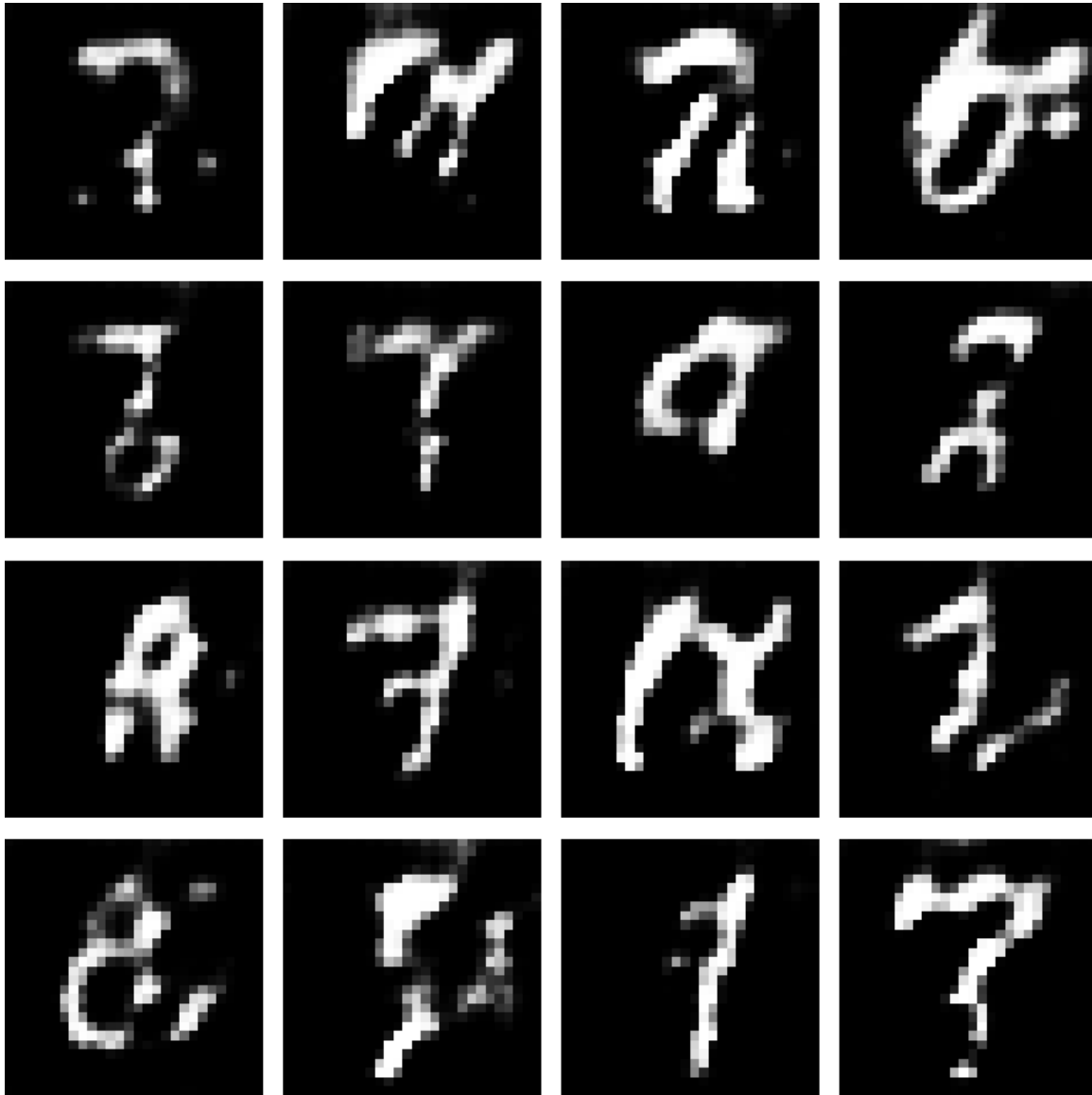
Step 14: Display the Generated Images

The synthetic handwritten digits generated by the trained Generator are displayed in a 4×4 grid.

```
plt.figure(figsize=(8, 8))

for i in range(num_examples):
    plt.subplot(4, 4, i + 1)
    plt.imshow(
        generated_images[i, :, :, 0] * 0.5 + 0.5,
        cmap="gray"
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Result

A Generative Adversarial Network (GAN) was successfully implemented using TensorFlow. The Generator learned to produce synthetic handwritten digit images from random noise vectors, while the Discriminator learned to distinguish between real and generated images. The trained Generator was able to generate realistic handwritten digit-like images after adversarial training.

Conclusion

In this practical, a Generative Adversarial Network (GAN) consisting of a Generator and a Discriminator was implemented. Both networks were trained simultaneously using the MNIST dataset. The Generator successfully learned the underlying distribution of handwritten digits and generated synthetic images from random noise, demonstrating the effectiveness of GANs for image generation tasks.

Practical 8: Write a program to implement a simple form of a recurrent neural network

Aim

To implement a Simple Recurrent Neural Network (RNN) using TensorFlow for predicting the quantity of rainfall based on the rainfall values of the previous four days.

Theory

A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data. Unlike feed-forward neural networks, an RNN retains information from previous inputs through its internal memory.

In this practical, a **4-to-1 RNN** is implemented. The model uses rainfall values from the previous four days to predict the rainfall of the next day.

Input Sequence:

Day 1 → Day 2 → Day 3 → Day 4

↓

Predict

↓

Day 5

RNNs are commonly used in:

- Weather forecasting
- Time series prediction
- Speech recognition
- Natural language processing

Solution

Step 1: Import Required Libraries

The required libraries are imported for data manipulation, visualization, model development, and evaluation.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.18.0

NumPy Version: 1.26.4

Step 2: Create the Rainfall Dataset

A sample rainfall dataset is created. Each value represents the rainfall (in millimeters) recorded on a particular day.

```
# Rainfall data (in millimetres)
rainfall = np.array([
    12, 15, 14, 18, 22,
    25, 24, 20, 18, 16,
    14, 12, 15, 19, 23,
    27, 26, 22, 20, 18,
    16, 15, 17, 21, 24,
    28, 30, 27, 25, 22
], dtype=np.float32)
```



```
# Normalize to the range 0-1
rainfall = rainfall / rainfall.max()

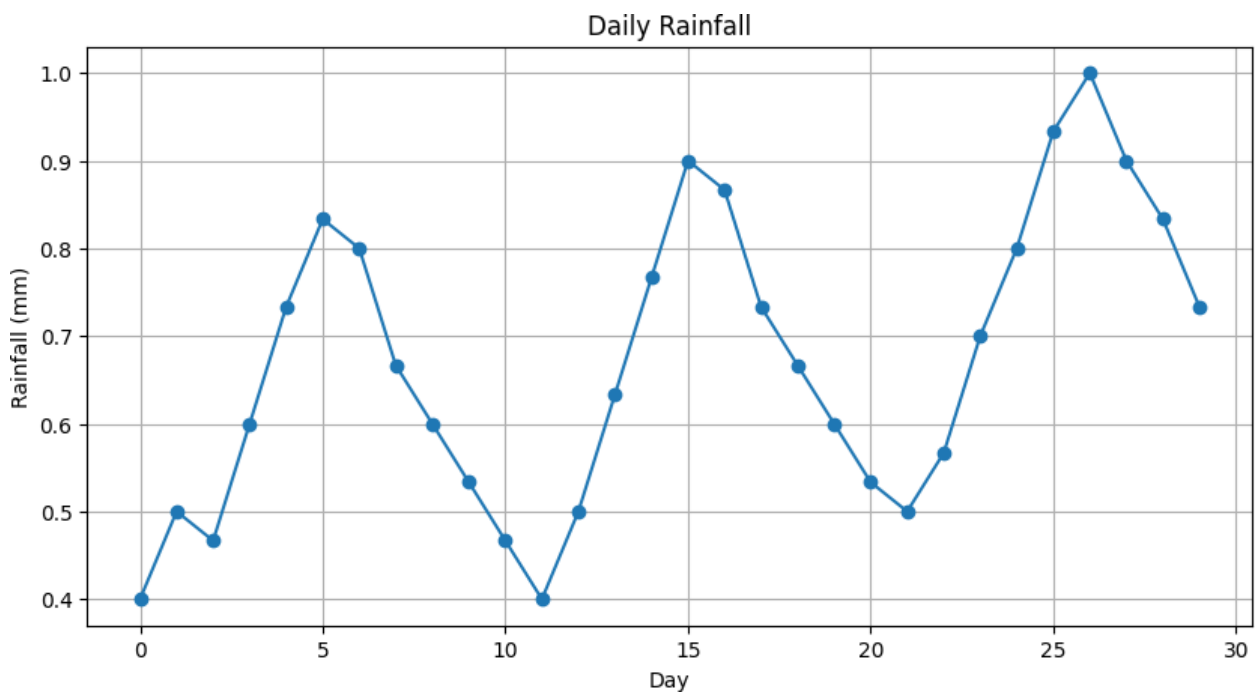
print(rainfall)
```

0.4	0.5	0.46666667	0.6	0.73333335	0.8333333
0.8	0.6666667	0.6	0.53333336	0.46666667	0.4
0.5	0.6333333	0.76666665	0.9	0.8666667	0.73333335
0.6666667	0.6	0.53333336	0.5	0.56666666	0.7
0.8	0.93333334	1.	0.9	0.8333333	0.73333335]

Step 3: Visualize the Rainfall Data

The rainfall values are plotted to observe the trend over the recorded days.

```
plt.figure(figsize=(10,5))
plt.plot(
    rainfall,
    marker='o'
)
plt.title("Daily Rainfall")
plt.xlabel("Day")
plt.ylabel("Rainfall (mm)")
plt.grid(True)
plt.show()
```



Step 4: Prepare the Input Sequences

The rainfall dataset is converted into input-output pairs using a sliding window approach.

Each input consists of rainfall values from four consecutive days, while the output is the rainfall value of the following day.

```
x = []
y = []

sequence_length = 4

for i in range(len(rainfall) - sequence_length):
    x.append(rainfall[i:i+sequence_length])
    y.append(rainfall[i+sequence_length])

x = np.array(x)
y = np.array(y)

print("Input Shape :", x.shape)
print("Output Shape:", y.shape)
```

Input Shape : (26, 4)
Output Shape: (26,)

Step 5: Reshape the Dataset

The input data is reshaped into three dimensions to match the input format required by the Simple RNN layer.

```
X = X.reshape((X.shape[0], X.shape[1], 1))  
  
print("Reshaped Input Shape:", X.shape)
```

Reshaped Input Shape: (26, 4, 1)

Step 6: Split the Dataset

The dataset is divided into training and testing sets using an 80:20 ratio.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    shuffle=False  
)  
  
print("Training Samples:", X_train.shape[0])  
print("Testing Samples :", X_test.shape[0])
```

Training Samples: 20 Testing
Samples : 6

Step 7: Build the Simple RNN Model

A Simple Recurrent Neural Network (RNN) is created using TensorFlow. The model consists of:

- One SimpleRNN layer with 16 neurons
- One Dense output layer with a single neuron

The SimpleRNN layer processes the sequence of rainfall values from the previous four days and predicts the rainfall for the next day.

```
model = tf.keras.Sequential([  
    tf.keras.layers.Input(shape=(4, 1)),  
    tf.keras.layers.SimpleRNN(16, activation="tanh"),  
    tf.keras.layers.Dense(1)  
)  
  
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
simple_rnn_2 (SimpleRNN)	(None, 16)	288
dense_2 (Dense)	(None, 1)	17

Total params: 305 (1.19 KB)

Trainable params: 305 (1.19 KB)

Non-trainable params: 0 (0.00 B)

Step 8: Compile the Model

The model is compiled using the Adam optimizer and Mean Squared Error (MSE) loss function.

Mean Squared Error is commonly used for regression problems where the objective is to predict continuous values such as rainfall.

```
model.compile(  
    optimizer="adam",  
    loss="mse",  
    metrics=["mae"]  
)  
  
print("Model compiled successfully.")
```

Model compiled successfully.

Step 9: Train the Model

The Simple RNN model is trained using the rainfall training dataset for 100 epochs.

During training, the model learns the relationship between rainfall values of the previous four days and the rainfall value of the following day.

```
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=4,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/100
4/4 ————— 1s 62ms/step - loss: 0.0727 - mae: 0.2465 - val_loss: 0.0235 - val_mae: 0.1278 Epoch 2/100
4/4 ————— 0s 16ms/step - loss: 0.0266 - mae: 0.1272 - val_loss: 0.0082 - val_mae: 0.0782 Epoch 3/100
4/4 ————— 0s 18ms/step - loss: 0.0181 - mae: 0.1255 - val_loss: 0.0164 - val_mae: 0.1172
...
Epoch 97/100
4/4 ————— 0s 18ms/step - loss: 0.0026 - mae: 0.0407 - val_loss: 0.0025 - val_mae: 0.0413 Epoch 98/100
4/4 ————— 0s 17ms/step - loss: 0.0026 - mae: 0.0405 - val_loss: 0.0028 - val_mae: 0.0435 Epoch 99/100
4/4 ————— 0s 15ms/step - loss: 0.0025 - mae: 0.0411 - val_loss: 0.0026 - val_mae: 0.0427 Epoch
100/100
4/4 ————— 0s 15ms/step - loss: 0.0026 - mae: 0.0411 - val_loss: 0.0025 - val_mae: 0.0411
```

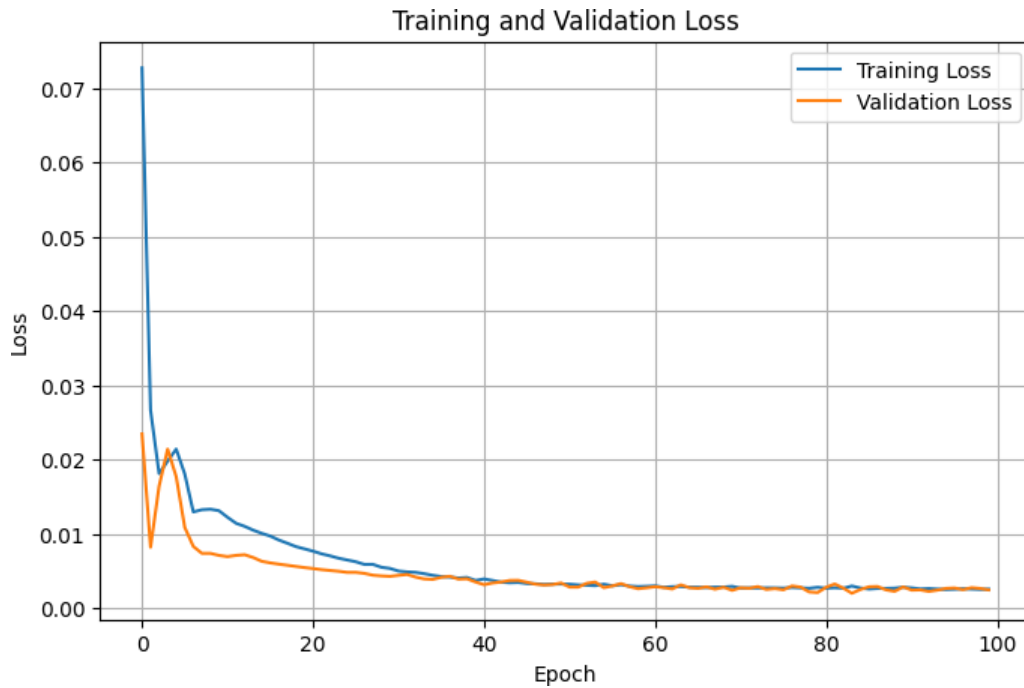
Step 10: Plot Training Loss

The graph below illustrates the training loss and validation loss over all training epochs.

```
plt.figure(figsize=(8,5))
plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```



Step 11: Evaluate the Model

The trained Simple RNN model is evaluated using the testing dataset. The Mean Squared Error (MSE) loss and Mean Absolute Error (MAE) are calculated to measure the prediction performance.

```
# Evaluate the model
loss, mae = model.evaluate(
    X_test,
    y_test,
    verbose=1
)

print(f"\nTest Loss (MSE): {loss:.4f}")
print(f"Test MAE      : {mae:.4f}")
```

```
0.1161 Test Loss (MSE): 0.0147
Test MAE      : 0.1161
```

Step 12: Predict Rainfall

The trained Simple RNN model predicts rainfall values for the testing dataset using the rainfall values from the previous four days.

```
# Predict rainfall
predictions = model.predict(X_test)

# Convert predictions back to millimetres
predictions = predictions * 30

print("Predicted Rainfall:")
print(predictions.flatten())
```

```
1/1
0s 99ms/step Predicted Rainfall:
[22.217249 24.388767 24.776285 24.17052 21.42637 18.124685]
```

Step 13: Compare Actual and Predicted Rainfall

The actual rainfall values are compared with the predicted rainfall values obtained from the Simple RNN model.

```
actual_values = y_test * 30
print("Actual\\tPredicted")

for actual, predicted in zip(actual_values, predictions.flatten()):
    print(f"{actual:.2f}\\t{predicted:.2f}")
```

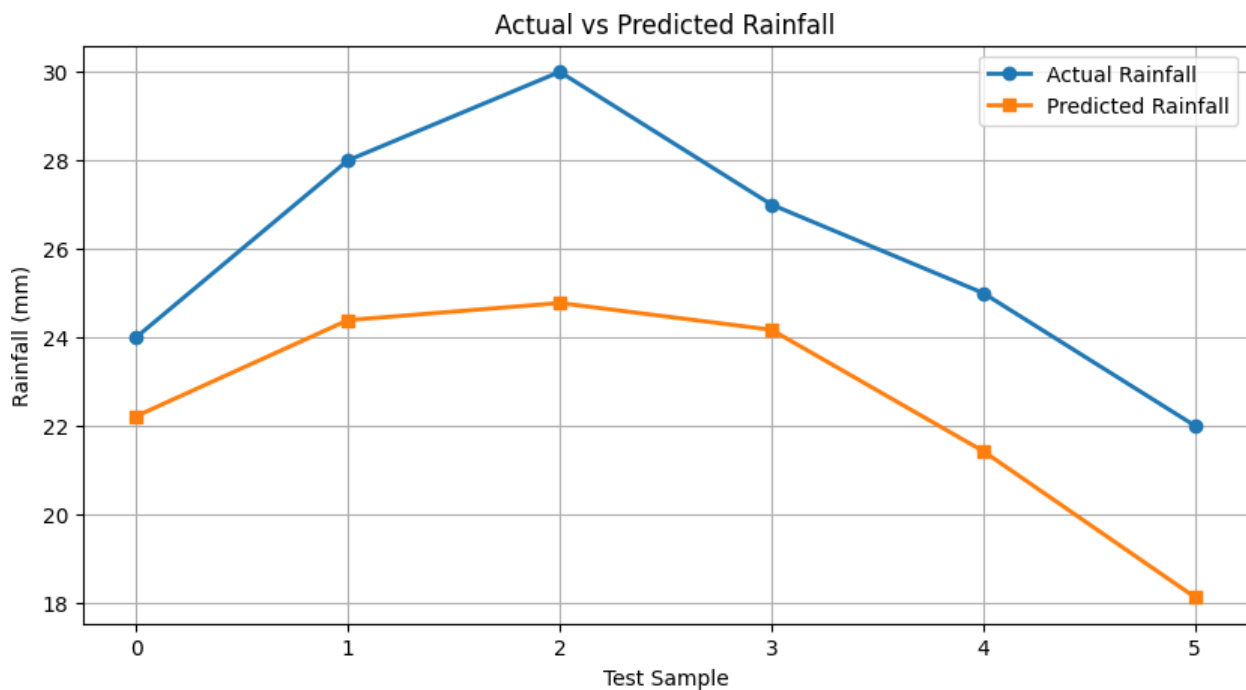
Actual	Predicted
24.00	22.22
28.00	24.39
30.00	24.78
27.00	24.17
25.00	21.43
22.00	18.12

Step 14: Plot Actual vs Predicted Rainfall

The graph below compares the actual rainfall values with the values predicted by the Simple RNN model.

```
# Convert actual values back to millimetres
actual_values = y_test * 30

plt.figure(figsize=(10, 5))
plt.plot(
    actual_values,
    marker='o',
    linewidth=2,
    label="Actual Rainfall"
)
plt.plot(
    predictions.flatten(),
    marker='s',
    linewidth=2,
    label="Predicted Rainfall"
)
plt.title("Actual vs Predicted Rainfall")
plt.xlabel("Test Sample")
plt.ylabel("Rainfall (mm)")
plt.legend()
plt.grid(True)
plt.show()
```



Step 15: Predict Rainfall for a New Sequence

The trained model is used to predict the rainfall for the next day using rainfall values from four consecutive previous days.

```
# Example: rainfall for four consecutive days
new_sequence = np.array([[22, 24, 26, 25]], dtype=np.float32)

# Normalize
new_sequence = new_sequence / 30
new_sequence = new_sequence.reshape((1, 4, 1))
predicted_rainfall = model.predict(new_sequence)
predicted_rainfall = predicted_rainfall * 30

print()
```

```
f"Predicted Rainfall for Next Day: "  
f"{predicted_rainfall[0][0]:.2f} mm"  
)
```

1/1 ————— 0s
23ms/step Predicted Rainfall for Next Day: 21.12 mm

Result

A Simple Recurrent Neural Network (4-to-1 RNN) was successfully implemented using TensorFlow to predict rainfall based on the rainfall values of the previous four days. The model learned temporal relationships in the sequential rainfall data and produced reasonable predictions on the testing dataset.

Conclusion

In this practical, a Simple Recurrent Neural Network (RNN) was developed for rainfall prediction using sequential data. The model demonstrated how information from previous time steps can be used to predict future values, highlighting the effectiveness of RNNs for time series forecasting problems such as weather prediction.

Practical 9: Implement LSTM for Sentiment Analysis on datasets like IMDB Movie Reviews Dataset or similar

Aim

To implement a Long Short-Term Memory (LSTM) neural network using TensorFlow for sentiment analysis of movie reviews from the IMDB dataset.

Theory

Sentiment Analysis is a Natural Language Processing (NLP) task used to determine whether a piece of text expresses a positive or negative opinion.

Long Short-Term Memory (LSTM) is a special type of Recurrent Neural Network (RNN) capable of learning long-term dependencies in sequential data. Unlike a Simple RNN, an LSTM can remember important information for longer periods using memory cells and gating mechanisms.

In this practical, an LSTM model is trained to classify movie reviews as either positive or negative.

Applications include:

- Product review analysis
- Social media sentiment analysis
- Customer feedback analysis
- Opinion mining

Solution

Step 1: Import Required Libraries

The required libraries are imported for data processing, visualization, and model development.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow Version:", tf.__version__)
print("NumPy Version:", np.__version__)
```

TensorFlow Version: 2.18.0
NumPy Version: 1.26.4

Step 2: Load the IMDB Dataset

The IMDB dataset is loaded from TensorFlow. Each movie review is represented as a sequence of integers, and each review is labeled as either positive (1) or negative (0).

```
# Keep only the 10,000 most frequent words
vocab_size = 10000

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.imdb.load_data(
    num_words=vocab_size
)

print("Training Samples:", len(x_train))
print("Testing Samples :", len(x_test))
```

Training Samples: 25000
Testing Samples : 25000

Step 3: Explore the Dataset

The length of the first review and its sentiment label are displayed to understand the structure of the dataset.

```
print("First Review Length:", len(x_train[0]))
```



```
print("First Review Label :", y_train[0])
```

First Review Length: 218
First Review Label : 1

Step 4: Pad the Sequences

All movie reviews are padded or truncated to a fixed length of 200 words to make them suitable for LSTM input.

```
max_length = 200
x_train = tf.keras.preprocessing.sequence.pad_sequences(
    x_train,
    maxlen=max_length
)
x_test = tf.keras.preprocessing.sequence.pad_sequences(
    x_test,
    maxlen=max_length
)

print("Training Shape:", x_train.shape)
print("Testing Shape :", x_test.shape)
```

Training Shape: (25000, 200)
Testing Shape : (25000, 200)

Step 5: Build the LSTM Model

The sentiment analysis model consists of:

- Embedding layer
- LSTM layer
- Dense output layer

The Embedding layer converts word indices into dense vector representations, while the LSTM layer learns sequential patterns in the reviews.

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(
        input_dim=vocab_size,
        output_dim=64,
        input_length=max_length
    ),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(
        1,
        activation="sigmoid"
    )
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Step 6: Compile the LSTM Model

The LSTM model is compiled using the Adam optimizer, Binary Crossentropy loss function, and Accuracy as the evaluation metric. Binary Crossentropy is suitable for binary classification problems such as positive and negative sentiment analysis.

```
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"]
```

```
)  
  
print("Model compiled successfully.")
```

Model compiled successfully.

Step 7: Train the LSTM Model

The model is trained using the IMDb training dataset. During training, the model learns patterns in movie reviews that indicate positive or negative sentiment.

```
history = model.fit(  
    x_train,  
    y_train,  
    validation_split=0.2,  
    epochs=5,  
    batch_size=64,  
    verbose=1  
)
```

Epoch 1/5
313/313 ————— 27s 82ms/step - accuracy: 0.7908 - loss: 0.4408 - val_accuracy: 0.8454 - val_loss: 0.3576 Epoch 2/5
313/313 ————— 25s 80ms/step - accuracy: 0.8999 - loss: 0.2521 - val_accuracy: 0.8640 - val_loss: 0.3249 Epoch 3/5
313/313 ————— 25s 80ms/step - accuracy: 0.9300 - loss: 0.1891 - val_accuracy: 0.8608 - val_loss: 0.3318 Epoch 4/5
313/313 ————— 25s 79ms/step - accuracy: 0.9474 - loss: 0.1446 - val_accuracy: 0.8652 - val_loss: 0.3561 Epoch 5/5
313/313 ————— 25s 79ms/step - accuracy: 0.9639 - loss: 0.1048 - val_accuracy: 0.8686 - val_loss: 0.4331

Step 8: Evaluate the Model

The trained LSTM model is evaluated using the testing dataset to measure its performance on unseen movie reviews.

```
loss, accuracy = model.evaluate(  
    x_test,  
    y_test,  
    verbose=1  
)  
  
print(f"\nTest Loss      : {loss:.4f}")  
print(f"Test Accuracy : {accuracy:.4f}")
```

782/782 ————— 13s 17ms/step - accuracy: 0.8524 - loss:

0.4748 Test Loss : 0.4748
Test Accuracy : 0.8524

Step 9: Predict Sentiment

The trained LSTM model predicts the sentiment of movie reviews from the testing dataset.

A prediction value greater than or equal to 0.5 indicates a positive review, while a value below 0.5 indicates a negative review.

```
predictions = model.predict(x_test[:10])  
  
for i, prediction in enumerate(predictions):  
    sentiment = "Positive" if prediction[0] >= 0.5 else "Negative"  
    print(  
        f"Review {i+1}: "  
        f"{prediction[0]:.4f} → {sentiment}"  
    )
```

1/1 —————
0s 115ms/step Review 1: 0.0450 → Negative
Review 2: 0.9994 → Positive
Review 3: 0.1809 → Negative
Review 4: 0.0696 → Negative
Review 5: 0.9999 → Positive
Review 6: 0.9277 → Positive
Review 7: 0.8042 → Positive
Review 8: 0.0019 → Negative
Review 9: 0.9905 → Positive
Review 10: 0.9836 → Positive

Step 10: Compare Actual and Predicted Sentiments

The predicted sentiment labels are compared with the actual sentiment labels from the testing dataset.

```
print("Review\tActual\tPredicted")

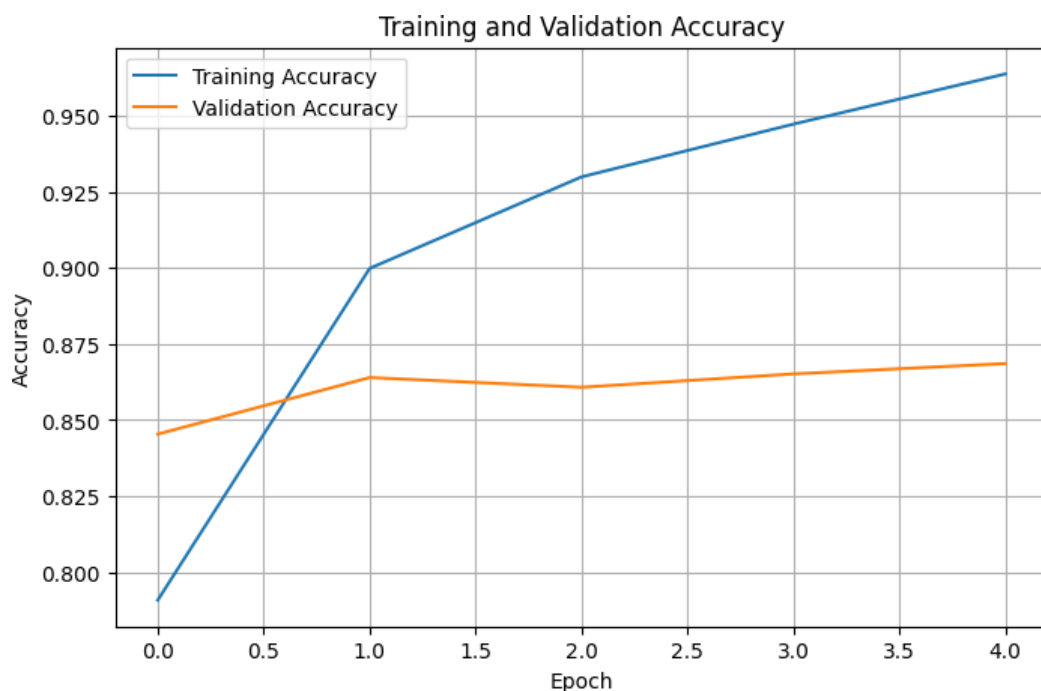
for i in range(10):
    predicted = 1 if predictions[i][0] >= 0.5 else 0
    print(
        f"{i+1}\t{y_test[i]}\t{predicted}"
    )
```

Review	Actual	Predicted
1	0	0
2	1	1
3	1	0
4	0	0
5	1	1
6	1	1
7	1	1
8	0	0
9	0	1
10	1	1

Step 11: Plot Training and Validation Accuracy

The graph below shows how the training accuracy and validation accuracy changed during the training process.

```
plt.figure(figsize=(8,5))
plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```



Step 12: Plot Training and Validation Loss

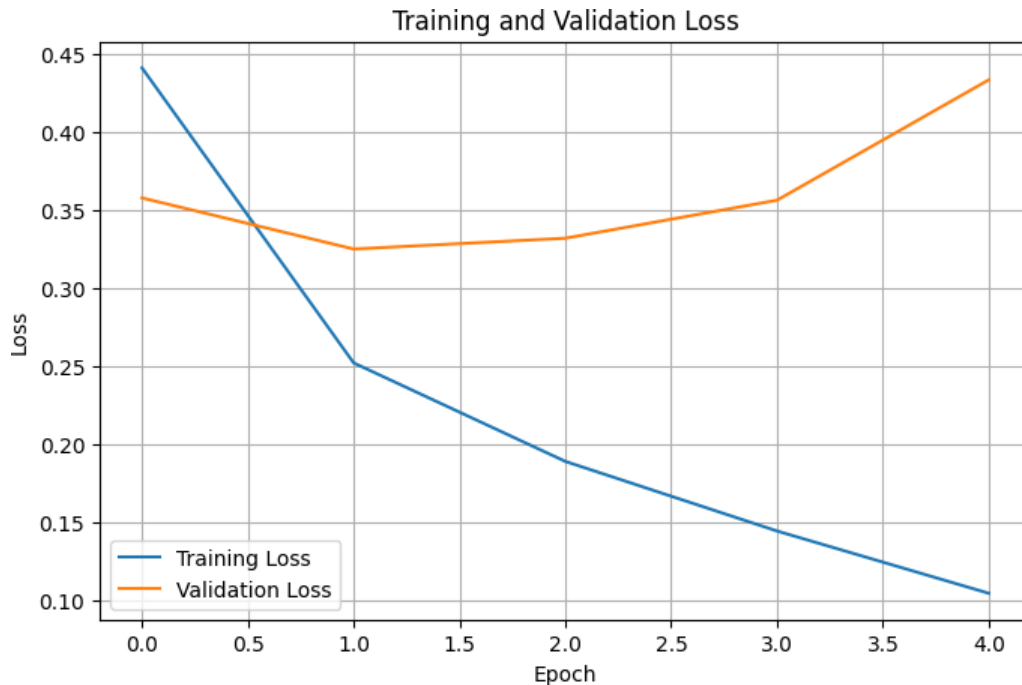
The graph below illustrates the training loss and validation loss during the learning process.

```
plt.figure(figsize=(8,5))
plt.plot(
    history.history["loss"],
```

```

    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

```



Step 13: Predict the Sentiment of a Sample Review

A movie review from the testing dataset is selected, and the trained LSTM model predicts whether the review expresses a positive or negative sentiment.

```

sample_review = x_test[0].reshape(1, max_length)
prediction = model.predict(sample_review)
sentiment = "Positive" if prediction[0][0] >= 0.5 else "Negative"
print(f"Predicted Probability : {prediction[0][0]:.4f}")
print(f"Predicted Sentiment : {sentiment}")
print(f"Actual Sentiment : {'Positive' if y_test[0] == 1 else 'Negative'}")

```

```

1/1
— 0s 117ms/step Predicted Probability : 0.0450
Predicted Sentiment : Negative Actual
Sentiment : Negative

```

Result

An LSTM-based sentiment analysis model was successfully implemented using TensorFlow and the IMDb movie review dataset. The trained model effectively classified movie reviews as positive or negative and achieved good accuracy on unseen test data.

Conclusion

In this practical, a Long Short-Term Memory (LSTM) neural network was implemented for sentiment analysis. The model learned sequential patterns in movie reviews and successfully classified sentiments into positive and negative categories. This practical demonstrated the effectiveness of LSTM networks in Natural Language Processing (NLP) tasks involving sequential text data.

Practical 10: Write a program for object detection from the image/ video

Aim

To implement object detection using TensorFlow and a pre-trained YOLOv8 model provided by KerasCV for detecting objects in an image.

Theory

Object Detection is a computer vision technique that detects and localizes multiple objects within an image or video.

Unlike image classification, object detection not only predicts the object category but also identifies its location using bounding boxes.

Modern object detection models such as YOLO (You Only Look Once), SSD (Single Shot Detector), and Faster R-CNN are trained on large datasets like COCO and can recognize many common object categories.

Applications include:

- Autonomous vehicles
- Video surveillance
- Robotics
- Smart traffic systems
- Face and pedestrian detection

Solution

Step 1: Import Required Libraries

Import libraries for deep learning, array manipulation, and image/video processing

```
import tensorflow as tf
import numpy as np
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

import pathlib
import os
```

Step 2: Create Images Folder

An images folder is created inside the current project directory. Sample images for object detection will be stored here.

```
project_dir = pathlib.Path(os.getcwd())
images_dir = project_dir / "images"
images_dir.mkdir(exist_ok=True)
```

Step 3: Download Sample Image

A sample image is downloaded into the images folder using a relative path. This makes the implementation operating system independent.

```
import urllib.request

image_url = "https://ultralytics.com/images/bus.jpg"
image_path = images_dir / "bus.jpg"

if not image_path.exists():
    urllib.request.urlretrieve(image_url, image_path)
```

Step 4: Display the Input Image

The downloaded image is displayed before object detection.

```

image = cv2.imread(str(image_path))
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10,8))
plt.imshow(image_rgb)
plt.axis("off")
plt.title("Input Image")

plt.show()

```

Input Image



Step 5: Load the Pre-trained YOLO Model

A pre-trained YOLO11 Nano model is loaded using the Ultralytics library. If the model is not already available on the system, it will be downloaded automatically during the first execution.

```

model = YOLO("yolo11n.pt")

print("YOLO model loaded successfully.")

YOLO model loaded successfully.

```

Step 6: Perform Object Detection

The input image is passed to the pre-trained YOLO model. The model detects objects present in the image and predicts their classes along with confidence scores and bounding boxes.

```

results = model.predict(
    source=str(image_path),
    conf=0.5,
    save=False,
    verbose=False
)

print("Object detection completed successfully.")

Object detection completed successfully.

```

Step 7: Display the Detection Summary

The detected objects along with their confidence scores are displayed.

```
result = results[0]

print("Detected Objects:\n")

for box in result.boxes:
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
    class_name = model.names[class_id]
    print(
        f"{class_name:15} Confidence: {confidence:.2f}"
    )
```

Detected Objects:

bus	Confidence: 0.94
person	Confidence: 0.89
person	Confidence: 0.88
person	Confidence: 0.86
person	Confidence: 0.62

Step 8: Display the Detection Results

The detected objects are displayed with bounding boxes and class labels drawn automatically by the YOLO model.

```
annotated_image = result.plot()

plt.figure(figsize=(12, 8))
plt.imshow(cv2.cvtColor(annotated_image, cv2.COLOR_BGR2RGB))
plt.title("Detected Objects")
plt.axis("off")

plt.show()
```

Detected Objects



Step 9: Save the Detection Result

The annotated image is saved in the images folder for future reference.

```
output_path = images_dir / "detected_bus.jpg"

cv2.imwrite(str(output_path), annotated_image)

print("Detection result saved at:")
```

Detection result saved at:

Step 10: Display the Number of Detected Objects

The total number of detected objects in the input image is displayed.

```
total_objects = len(result.bboxes)

print(f"Total Objects Detected: {total_objects}")
```

Total Objects Detected: 5

Result

A pre-trained YOLO object detection model was successfully implemented to detect and localize multiple objects in an input image. The model correctly identified objects, displayed their class labels with confidence scores, and drew bounding boxes around each detected object. The detected image was successfully saved. The same model can also be used for real-time object detection using a webcam or video.

Conclusion

In this practical, object detection was implemented using a pre-trained YOLO deep learning model. Unlike image classification, object detection identifies both the class and the location of multiple objects in an image. The practical demonstrated how deep learning models can perform accurate and efficient object detection, making them suitable for applications such as surveillance, autonomous driving, robotics, and smart monitoring systems.